



Université de Montpellier

Master Informatique

Intelligence Artificielle et Science des Données

Rapport de TER

Trouvez le bon profil : analyse automatique de
CV avec les modèles de langage.

Groupe : GBL-ALM

Réalisé par :

Nada Abir BOUHADJAR | 2512053

Louiza GACI | 2515409

Mohand GUESSAS | 22423442

Aymene LAZREG | 22401059

Encadré par :

Pascal PONCELET

Année universitaire : 2025 – 2026

Table des matières

Résumé	6
Introduction générale	7
1 Chapitre 1 : Notions préliminaires	9
1.1 Intelligence artificielle et apprentissage automatique	9
1.1.1 Intelligence artificielle	9
1.1.2 Apprentissage automatique	10
1.2 Apprentissage profond	10
1.2.1 Architecture d'un réseau de neurones	10
1.2.2 Représentation mathématique d'un neurone	11
1.2.3 Fonctions d'activation	11
1.2.4 Processus d'apprentissage d'un réseau de neurones	11
1.3 Traitement automatique du langage naturel	12
1.3.1 Pipeline classique du traitement de texte	12
1.4 Représentation vectorielle du texte (embeddings)	13
1.4.1 Word Embeddings	13
1.4.2 Similarité cosinus	13
1.5 Le mécanisme d'attention et les Transformers	14
1.5.1 La self-attention	14
1.5.2 L'attention multi-têtes (Multi-Head Attention)	15
1.5.3 Les Transformers	15
1.5.3.1 Architecture d'un Transformer	15
1.5.3.2 Entraînement des Transformers	16
1.5.3.3 Des Transformers aux sentence embeddings.	16
1.6 Conclusion	16
2 Chapitre 2 : Extraction et nettoyage des CV au format PDF	18
2.1 Problématique	18
2.2 Choix techniques	18
2.3 Méthode proposée	19
2.4 Gestion des cas particuliers	21
2.5 Limites	21

2.6	Conclusion	21
3	Chapitre 3 : Méthodologie de comparaison automatique CVs / Offres d'emploi	22
3.1	Modèle d'embeddings retenu	22
3.2	Le problème de la taille des entrées	22
3.3	Stratégies de chunking explorées	23
3.3.1	Découpage par sauts de ligne (<code>chunk_by_newlines</code>)	23
3.3.1.1	Avantages et inconvénients	24
3.3.1.2	Exemple expérimental	24
3.3.2	Découpage par mots fixes avec chevauchement (<code>chunk_by_words</code>)	25
3.3.2.1	Avantages et inconvénients	25
3.3.2.2	Exemple expérimental	25
3.3.3	Découpage par phrases (<code>chunk_by_sentences</code>)	26
3.3.3.1	Avantages et inconvénients	26
3.3.3.2	Exemple expérimental	27
3.3.4	Chunking sémantique (<code>chunk_semantic</code>)	27
3.3.4.1	Avantages et inconvénients	28
3.3.4.2	Exemple expérimental	28
3.3.5	Fusion des stratégies de chunking (<code>chunk_fusion</code>)	29
3.3.5.1	Avantages et inconvénients	29
3.3.5.2	Exemple expérimental	29
3.3.6	Chunking hiérarchique adaptatif (<code>chunk_adaptatif</code>)	30
3.3.6.1	Avantages et inconvénients	32
3.3.6.2	Exemple expérimental	32
3.4	Protocole d'évaluation	33
3.4.1	Jeu de données de test	33
3.4.2	Métriques d'évaluation	33
3.5	Résultats et discussion	34
3.6	Conclusion	35
4	Chapitre 4 : Interface web et intégration	36
4.1	Architecture générale de l'application	36
4.2	Diagramme de cas d'utilisation	37
4.3	Implémentation du backend avec Flask	38

4.3.1	Architecture du serveur	38
4.3.2	Endpoints exposés	38
4.3.3	Gestion des fichiers uploadés	38
4.3.4	Intégration du moteur IA	39
4.4	Interface utilisateur	39
4.4.1	Structure de la page	39
4.4.2	Comportement dynamique – JavaScript	40
4.4.3	Fiche détaillée du candidat	40
4.4.4	Visualisation du CV complet	42
4.5	Extraction automatique d’informations	42
4.6	Tests et validation	43
4.6.1	Environnement de test	43
4.6.2	Scénarios fonctionnels	43
4.7	Conclusion	44

Conclusion générale	45
----------------------------	-----------

Table des figures

1	Relation entre l'intelligence artificielle, l'apprentissage automatique et l'apprentissage profond. [1]	9
2	Un exemple de réseau de neurones artificiel. [2]	11
3	Visualisation du processus d'entraînement d'un réseau de neurones. [3]	12
4	Similarité cosinus entre les mots <i>développeur</i> , <i>programmeur</i> et <i>tomate</i> . [4]	14
5	Illustration du calcul de l'attention à partir des vecteurs Query, Key et Value. [5]	15
6	Pipeline général d'extraction et de nettoyage des CV [14].	19
7	Comparaison des résultats d'extraction entre l'approche brute et notre méthode.	20
8	Pipeline matching CV/offre (créé avec lucid.app) [14]	23
9	Pipeline du découpage par sauts de ligne (<code>chunk_by_newlines</code>) [14]	23
10	Pipeline du découpage par mots fixes avec chevauchement [14]	25
11	Pipeline du découpage par phrases [14]	26
12	Pipeline du découpage sémantique [14]	27
13	Pipeline de la fusion des stratégies de chunking [14]	29
14	Pipeline global de la méthode de découpage adaptatif [14]	31
15	Détails du découpage adaptatif [14]	31
16	Pipeline de la fusion sémantique des blocs [14]	32
17	Architecture générale de l'application web.	36
18	Diagramme de cas d'utilisation – RecruitAI.	37
19	Formulaire d'analyse : saisie de l'offre d'emploi et upload des CVs.	39
20	Zone paramètres et résultats.	40
21	Visualisation du résultat de classement	41
22	Fiche détaillée d'un candidat.	41
23	Modale de visualisation du CV complet.	42
24	Exemple de classement obtenu après analyse des CVs.	43

Liste des tableaux

1	Avantages et inconvénients du découpage par sauts de ligne	24
2	Avantages et inconvénients du découpage par mots fixes avec chevauchement . .	25
3	Avantages et inconvénients du découpage par phrases	26
4	Avantages et inconvénients du chunking sémantique	28
5	Avantages et inconvénients de la fusion des stratégies de chunking	29
6	Scores par stratégie et poids associés	30
7	Avantages et inconvénients du chunking hiérarchique adaptatif	32
8	Comparaison des six stratégies de chunking sur cinq métriques d'évaluation. Les meilleures valeurs par métrique sont indiquées en gras.	34
9	Résultats des tests fonctionnels.	43

Résumé

Ce rapport présente le développement d'un système automatique de matching entre des CVs et des offres d'emploi, utilisant des méthodes avancées de traitement automatique du langage naturel (NLP). L'approche repose sur la vectorisation des textes via des embeddings multilingues (paraphrase-multilingual-MiniLM-L12-v2) et le calcul de similarité cosinus.

Une première étape consiste à extraire et nettoyer le texte des CVs au format PDF, en gérant les mises en page complexes (colonnes, puces, barres de progression). Face à la limite de taille des modèles d'embeddings, nous explorons et comparons six stratégies de chunking (découpage en blocs), allant d'approches simples (sauts de ligne, mots fixes) à des méthodes plus sophistiquées (chunking sémantique, fusion pondérée, découpage hiérarchique adaptatif).

Les résultats expérimentaux montrent que la stratégie adaptative est la plus robuste, obtenant un NDCG de 0.9789 et une Precision@3 parfaite de 1.00. Une interface web développée avec Flask permet aux recruteurs d'utiliser facilement le système : import de l'offre, upload des CVs, classement dynamique et inspection détaillée des candidats.

Mots-clés : NLP, embeddings, similarité cosinus, chunking, Transformers, Flask, matching CV/offre.

Introduction générale

Contexte

Le recrutement est un processus essentiel pour toute entreprise, mais il devient de plus en plus chronophage face au volume croissant de candidatures reçues pour chaque offre d'emploi. Les recruteurs doivent parcourir manuellement des dizaines, voire des centaines de CVs pour identifier les profils les plus pertinents. Cette tâche, répétitive et fastidieuse, laisse une place importante à la subjectivité et peut conduire à des oublis de candidats qualifiés.

Parallèlement, les avancées récentes en traitement automatique du langage naturel (NLP) offrent de nouvelles opportunités pour automatiser l'analyse de documents textuels. Les modèles basés sur les Transformers, comme sentence-transformers, permettent de produire des embeddings contextuels. Dans ce projet, nous utilisons un modèle de sentence embedding (paraphrase-multilingual-MiniLM-L12-v2) qui génère un vecteur unique pour chaque texte, facilitant ainsi la comparaison sémantique entre CVs et offres d'emploi.

Problématique

Comment concevoir un système capable de comparer automatiquement des CVs à une offre d'emploi et de classer les candidats par ordre de pertinence sémantique ?

Cette problématique soulève plusieurs défis techniques :

- L'extraction de texte exploitable à partir de CVs au format PDF, souvent structurés en colonnes et contenant des éléments visuels (icônes, barres de progression).
- La limitation de taille des modèles d'embeddings, qui ne peuvent traiter des documents longs comme des CVs en une seule passe.
- Le choix d'une stratégie de découpage (chunking) optimale pour préserver la cohérence sémantique des blocs.
- La nécessité de fournir une interface accessible aux recruteurs non techniques.

Objectifs

Ce projet de TER poursuit plusieurs objectifs :

1. Extraire et nettoyer le texte de CVs PDF, en gérant les mises en page complexes et les cas particuliers (documents scannés).
2. Implémenter et comparer six stratégies de chunking pour déterminer la plus adaptée au matching CV/offre.
3. Développer un classement basé sur les embeddings et la similarité cosinus.
4. Proposer une interface web intuitive permettant aux recruteurs d'utiliser le système sans compétences techniques.

Plan du rapport

Le rapport est organisé en quatre chapitres. Le Chapitre 1 présente les notions préliminaires nécessaires à la compréhension du projet (IA, NLP, embeddings, Transformers). Le Chapitre

2 détaille l'extraction et le nettoyage des CVs PDF. Le Chapitre 3 expose la méthodologie de comparaison et les résultats expérimentaux des six stratégies de chunking. Enfin, le Chapitre 4 décrit l'interface web développée et son intégration.

1 Chapitre 1 : Notions préliminaires

Il est essentiel de connaître les notions de base pour analyser et comparer automatiquement des documents textuels. L'objectif du projet est d'identifier automatiquement les correspondances entre des CV et des offres d'emploi à l'aide de méthodes d'intelligence artificielle.

Ce chapitre présente les notions essentielles mobilisées dans la suite du projet : apprentissage automatique, apprentissage profond, traitement automatique du langage, représentations vectorielles du texte, similarité cosinus et Transformers. Ces notions servent ensuite à justifier les choix techniques retenus pour le système de matching.

1.1 Intelligence artificielle et apprentissage automatique

1.1.1 Intelligence artificielle

L'intelligence artificielle (IA) est un domaine de l'informatique qui vise à concevoir des systèmes capables de réaliser des tâches qui nécessitent normalement l'intelligence humaine. L'objectif est de permettre aux machines de traiter des informations, de prendre des décisions et de résoudre des problèmes complexes de manière autonome.

Parmi les exemples concrets d'applications de l'IA, on peut citer :

- La reconnaissance d'images, utilisée dans la détection d'objets.
- Les assistants virtuels ou chatbots.
- L'analyse automatique de documents, comme dans le cadre de notre projet où des CV sont comparés à des offres d'emploi pour identifier les meilleurs candidats.

L'IA englobe plusieurs sous-domaines. Comme le montre la figure 1, l'apprentissage automatique est un sous-domaine de l'IA, et l'apprentissage profond constitue lui-même une branche de l'apprentissage automatique.

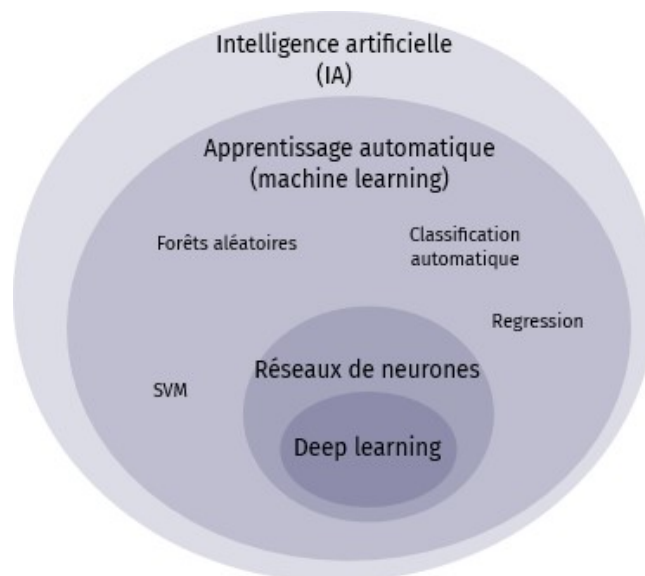


FIGURE 1 – Relation entre l'intelligence artificielle, l'apprentissage automatique et l'apprentissage profond. [1]

1.1.2 Apprentissage automatique

L'apprentissage automatique (Machine Learning) est un sous-domaine de l'intelligence artificielle où les ordinateurs apprennent à partir de données au lieu d'être programmés explicitement. L'ordinateur identifie des motifs dans les données pour prendre des décisions ou faire des prédictions. Contrairement à la programmation traditionnelle où les règles sont définies par l'humain, le machine learning permet à l'ordinateur de découvrir lui-même les règles à partir d'exemples.

L'apprentissage automatique comporte généralement deux étapes essentielles :

1. **Phase d'entraînement** : le modèle apprend à partir d'un jeu de données d'exemples, en ajustant ses paramètres pour minimiser les erreurs.
2. **Phase de prédiction** : le modèle utilise ce qu'il a appris pour faire des prédictions ou des décisions sur de nouvelles données.

Dans le cadre de ce projet, ces notions d'apprentissage automatique constituent principalement un cadre général permettant d'introduire l'apprentissage profond et les méthodes de traitement du texte utilisées par la suite.

1.2 Apprentissage profond

L'apprentissage profond (Deep Learning) est un sous-domaine du machine learning qui utilise des réseaux de neurones artificiels à plusieurs couches pour apprendre automatiquement des représentations complexes à partir de grandes quantités de données. Inspirés du cerveau humain, les réseaux de neurones artificiels traitent l'information via des neurones interconnectés : chaque neurone reçoit des entrées, effectue un calcul, puis transmet le résultat aux neurones suivants.

1.2.1 Architecture d'un réseau de neurones

Un réseau de neurones comprend généralement trois types de couches :

- **La couche d'entrée (Input Layer)** : reçoit les données initiales.
- **Les couches cachées (Hidden Layers)** : transforment les données pour extraire des caractéristiques pertinentes.
- **La couche de sortie (Output Layer)** : produit le résultat final du modèle.

La figure 2 illustre l'architecture générale d'un réseau de neurones artificiel. À gauche, en jaune, nous trouvons la couche d'entrée composée ici de 3 neurones. À droite, en rose, il s'agit de la couche de sortie composée d'un seul neurone. Enfin, ce réseau contient deux couches cachées, en bleu et en vert, composées de 4 neurones chacune.

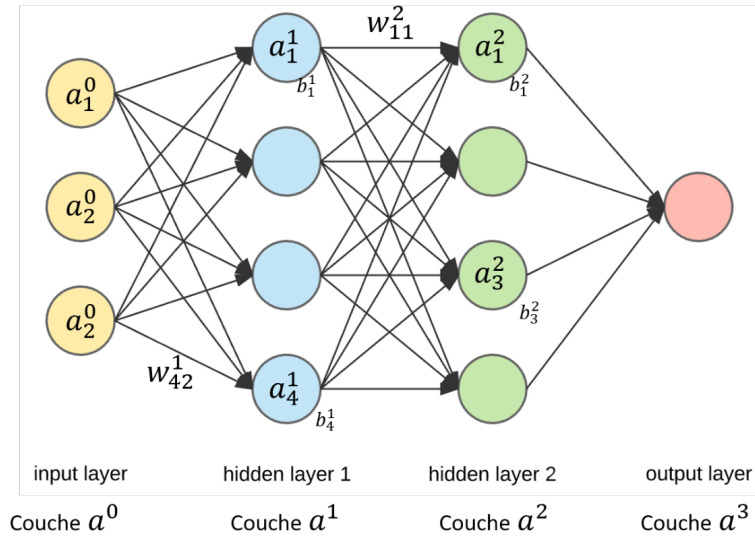


FIGURE 2 – Un exemple de réseau de neurones artificiel. [2]

1.2.2 Représentation mathématique d'un neurone

Un neurone artificiel reçoit plusieurs entrées et calcule une combinaison pondérée pour produire une sortie. Plus formellement :

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right) \quad (1)$$

où x_i sont les entrées, w_i les poids, b le biais, f la fonction d'activation, et y la sortie du neurone.

Les poids et le biais sont des paramètres qui sont ajustés durant la phase d'apprentissage afin de permettre au modèle de produire des prédictions de plus en plus précises.

1.2.3 Fonctions d'activation

Chaque couche contient une fonction d'activation qui transforme la combinaison linéaire des entrées en sortie non linéaire, permettant ainsi au neurone d'apprendre des relations complexes.

Parmi les fonctions d'activation les plus couramment utilisées, on peut citer notamment Sigmoid, ReLU et Leaky ReLU.

Le choix de la fonction d'activation dépend du contexte d'utilisation du modèle et contribue à la capacité des réseaux de neurones à modéliser des relations complexes.

1.2.4 Processus d'apprentissage d'un réseau de neurones

L'apprentissage d'un réseau de neurones consiste à ajuster les paramètres du modèle afin de réduire l'erreur entre la sortie produite par le réseau et la valeur attendue. Ce processus repose principalement sur deux étapes : la propagation avant et la rétropropagation. La figure 3 illustre ces deux étapes principales du processus d'entraînement.

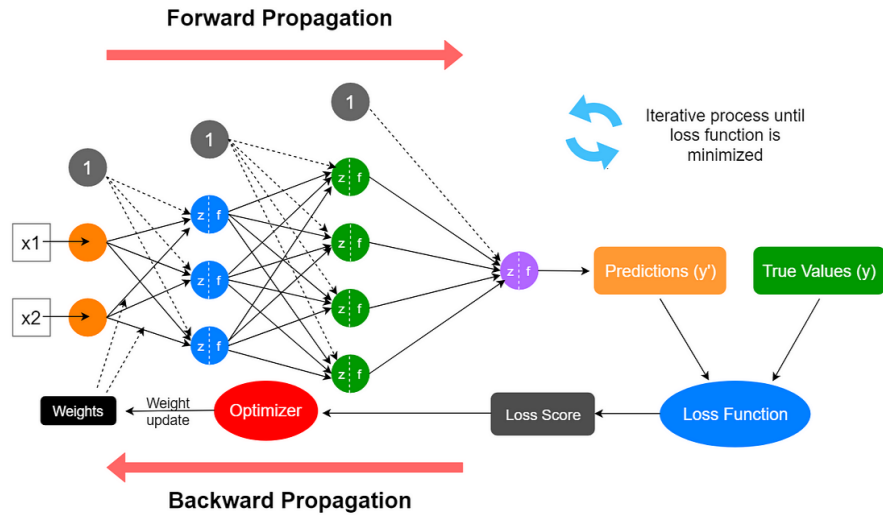


FIGURE 3 – Visualisation du processus d’entraînement d’un réseau de neurones. [3]

- **Propagation avant (forward propagation)** : les données traversent successivement les couches du réseau et chaque neurone calcule sa sortie, jusqu’à obtenir la prédiction finale.
- **Rétropropagation (backpropagation)** : l’erreur entre la prédiction $\hat{y}_i$ et la valeur réelle y_i est calculée à l’aide d’une fonction de coût. Cette erreur est ensuite propagée en sens inverse pour ajuster les poids du réseau.

Ces étapes sont répétées sur un grand nombre d’exemples jusqu’à ce que le modèle atteigne des performances satisfaisantes.

1.3 Traitement automatique du langage naturel

Le NLP est un domaine de l’intelligence artificielle qui permet aux ordinateurs de comprendre, interpréter et manipuler le langage humain, souvent ambigu et complexe. Il combine plusieurs disciplines : linguistique (structures grammaticales), informatique et machine learning (modélisation et apprentissage) et statistiques (analyse de grandes quantités de texte). Il est utilisé dans de très nombreuses applications, telles que la traduction automatique, les assistants virtuels et l’analyse de sentiment par exemple

1.3.1 Pipeline classique du traitement de texte

Jusqu’aux années 2010, le NLP reposait généralement sur une approche appelée “sac de mots”, intégrée dans un pipeline d’étapes successives visant à préparer le texte avant l’application d’un modèle d’apprentissage automatique. Ces étapes incluaient notamment :

- **Tokenisation** : découpage du texte en unités (mots, phrases ou caractères).
- **Nettoyage** : suppression de la ponctuation, des caractères spéciaux et des mots très fréquents (stop words) pour ne garder que l’information pertinente.
- **Lemmatisation / Stemming** : réduction des mots à leur forme de base pour regrouper les différentes variantes. Par exemple :
 - développeur, développeurs → développeur
 - analysant, analyse, analysé → analyser

- **Vectorisation** : transformation des mots ou documents en vecteurs numériques exploitables par les algorithmes d'apprentissage automatique pour la classification ou la mesure de similarité.

Même si ces approches se sont révélées très efficaces, elles souffrent de deux problèmes importants. D'une part l'ordre des mots dans une phrase n'est pas respectée. Par exemple, les deux phrases suivantes : "Le candidat a dirigé une équipe" et "Une équipe a dirigé le candidat" seront analysées de la même manière alors qu'elles n'ont pas du tout le même sens. D'autre part, aucune "sémantique" n'est associée aux mots et, par exemple, les mots "programmeur" et "développeur" sont traités comme des mots distincts sans relation particulière, au même titre que "développeur" et "tomate".

1.4 Représentation vectorielle du texte (embeddings)

Comme nous venons de le voir avec les approches de type sac de mots, il est difficile de représenter les relations sémantiques entre les mots. Pour pallier ce problème, de nouvelles approches ont été proposées, comme Word2Vec [6] ou GloVe [7], afin de représenter les mots sous la forme de vecteurs appelés *embeddings*.

1.4.1 Word Embeddings

Les word embeddings offrent une représentation vectorielle de dimension réduite, dans laquelle les mots ayant un sens similaire sont proches dans l'espace vectoriel. Ces représentations permettent aux modèles de capturer les relations sémantiques entre les mots.

Des approches comme Word2Vec [6] ou GloVe [7] ont marqué une étape importante dans la représentation vectorielle des mots. Elles sont mentionnées ici comme repères, avant d'introduire des méthodes plus récentes, mieux adaptées à la comparaison sémantique de documents dans notre système.

1.4.2 Similarité cosinus

Une fois les textes représentés par des vecteurs, leur proximité peut être mesurée mathématiquement à l'aide de la similarité cosinus. Cette métrique calcule le cosinus de l'angle entre deux vecteurs A et B

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} \quad (2)$$

La valeur de la similarité cosinus varie généralement entre :

- **1** : les vecteurs sont orientés dans la même direction, ce qui traduit une forte proximité entre les textes représentés.
- **0** : les vecteurs sont orthogonaux (textes indépendants).
- **-1** : les vecteurs sont diamétralement opposés.

La figure 4 illustre ce principe sur un exemple simple : les mots *développeur* et *programmeur* sont plus proches sémantiquement que le mot *tomate*.

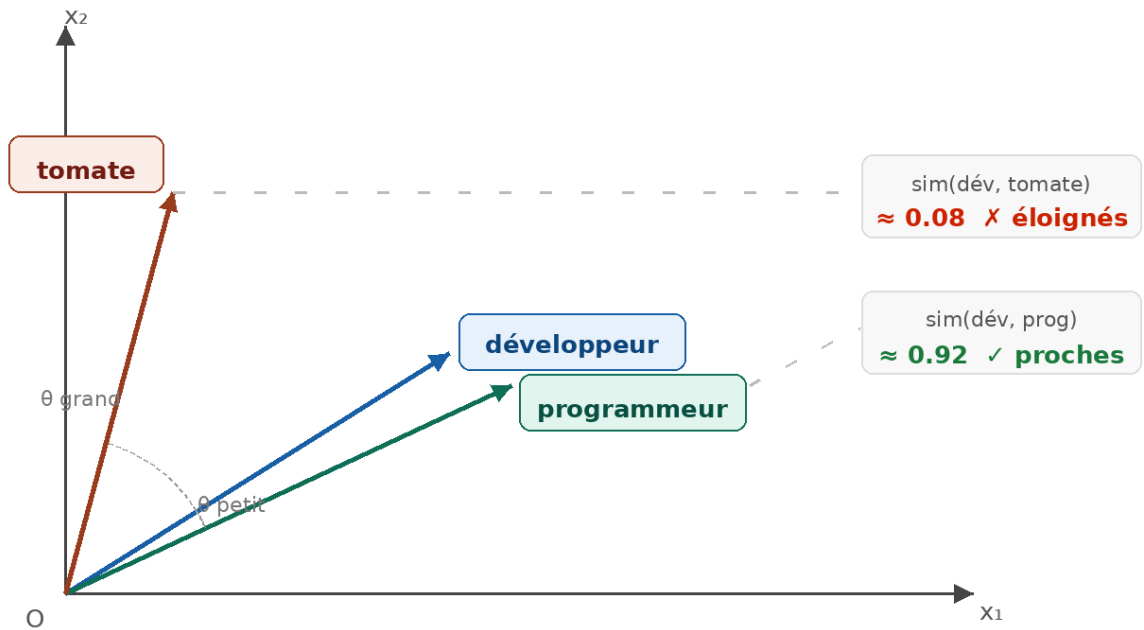


FIGURE 4 – Similarité cosinus entre les mots *développeur*, *programmeur* et *tomate*. [4]

Bien que la similarité cosinus permette de comparer efficacement des vecteurs, les embeddings fixes présentent des limites : un mot conserve le même vecteur quel que soit son contexte. Pour pallier cette limite, les modèles fondés sur l'attention, notamment les Transformers, produisent des embeddings contextualisés.

1.5 Le mécanisme d'attention et les Transformers

Les embeddings fixes comme Word2Vec ou GloVe ne tiennent pas compte du contexte d'un mot. Pour surmonter cette limite, les Transformers [8] utilisent le mécanisme d'attention pour produire des embeddings contextuels, où chaque vecteur dépend de l'ensemble de la phrase ou du document.

1.5.1 La self-attention

Chaque mot considère les autres mots de la séquence pour construire sa représentation contextuelle. Dans les Transformers [8], chaque mot est projeté en trois vecteurs :

- **Query (Q)** : représente le mot qui cherche des informations.
- **Key (K)** : représente les informations de référence contenues dans chaque mot.
- **Value (V)** : représente le contenu sémantique associé à chaque mot.

Le fonctionnement du mécanisme d'attention se déroule en trois étapes clés :

1. **Calcul des scores d'affinité** : la Query d'un mot est comparée aux Keys des autres mots de la séquence pour calculer un score d'affinité.
2. **Normalisation (Softmax)** : ces scores sont ensuite passés dans une fonction Softmax, qui les transforme en poids normalisés.

3. **Agrégation des informations** : ces poids servent à combiner les vecteurs Value, produisant une nouvelle représentation du mot qui prend en compte les mots les plus pertinents de son contexte.

L'attention est calculée par :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (3)$$

La figure 5 illustre ce calcul de l'attention à partir des vecteurs Query, Key et Value.

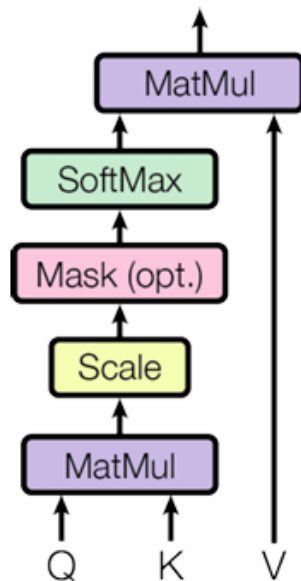


FIGURE 5 – Illustration du calcul de l'attention à partir des vecteurs Query, Key et Value. [5]

Ce mécanisme permet de capturer les relations entre les mots d'un texte.

1.5.2 L'attention multi-têtes (Multi-Head Attention)

L'attention multi-têtes consiste à effectuer plusieurs mécanismes d'attention en parallèle, chacun capturant un type de relation différent entre les mots. Les résultats sont ensuite combinés pour produire une représentation finale plus riche.

Cette approche permet de mieux modéliser des relations complexes et d'améliorer la qualité des représentations contextuelles.

1.5.3 Les Transformers

Les Transformers sont des architectures de réseaux de neurones entièrement fondées sur le mécanisme d'attention, sans recours aux réseaux récurrents. Ils sont aujourd'hui à la base des modèles de traitement du texte les plus performants.

1.5.3.1 Architecture d'un Transformer

Un Transformer typique se compose de blocs empilés comprenant :

- une couche de *multi-head self-attention* ;
- un réseau *feed-forward* ;
- des connexions résiduelles avec normalisation (*Add et Norm*).

Ces blocs permettent de transformer les vecteurs d'entrée en représentations contextualisées, exploitables pour la classification, la similarité ou la génération de texte.

Les architectures modernes peuvent reposer sur un encodeur seul, un décodeur seul, ou un couple encodeur-décodeur. Par exemple, BERT suit une architecture encodeur seul, ce qui lui permet de produire des représentations riches d'un texte entier. GPT appartient à la famille des modèles décodeur seul, orienté vers la génération de texte. T5, quant à lui, suit une architecture encodeur-décodeur.

1.5.3.2 Entraînement des Transformers

Leur apprentissage suit les étapes classiques du deep learning : propagation avant, calcul de la perte, rétropropagation et mise à jour des paramètres. En étant entraînés sur de grandes quantités de données, ils apprennent des représentations riches et contextualisées du langage.

1.5.3.3 Des Transformers aux sentence embeddings.

Pour notre système de matching, ce qui nous intéresse n'est pas de générer du texte, mais de représenter un CV ou une offre d'emploi par un vecteur unique, comparable à d'autres vecteurs. Des modèles comme paraphrase-multilingual-MiniLM-L12-v sont des Transformers fine-tunés spécifiquement pour produire des sentence embeddings : un vecteur dense qui capture le sens global d'un texte entier, et non mot par mot.

C'est précisément ce type de représentation que nous utilisons dans notre système : chaque CV et chaque offre d'emploi est transformé en un vecteur par ce modèle, puis la similarité cosinus entre ces vecteurs permet de mesurer la proximité sémantique entre un profil et une offre.

1.6 Conclusion

Ce chapitre a présenté les notions fondamentales nécessaires à l'analyse automatique de textes : l'intelligence artificielle et l'apprentissage automatique comme cadre général, le traitement automatique du langage naturel, les représentations vectorielles du texte et la similarité cosinus, puis les Transformers et le mécanisme d'attention.

Dans ce projet, nous utilisons principalement des embeddings pour représenter les CV et les offres d'emploi sous forme de vecteurs denses, permettant de capturer leur contenu sémantique. Contrairement aux approches classiques comme le Sac des mots ou TF-IDF, qui se limitent à compter les occurrences de mots sans tenir compte de leur sens, les embeddings permettent de représenter le sens global d'un texte entier dans un espace vectoriel. Ainsi, deux textes sémantiquement proches, même s'ils n'utilisent pas exactement les mêmes mots, se retrouveront proches dans cet espace.

Ces vecteurs sont ensuite comparés à l'aide de la similarité cosinus, qui mesure l'angle entre deux vecteurs. Dans notre système, plus ce score est élevé, plus le CV et l'offre d'emploi sont sémantiquement proches.

Par ailleurs, les documents textuels tels que les CV peuvent être longs et dépasser la taille maximale acceptée par les modèles d’embeddings. Pour pallier cette limite, nous recourons au chunking, qui consiste à découper chaque document en blocs de taille fixe. Cette approche permet non seulement de respecter les contraintes des modèles, mais aussi d’améliorer le matching en comparant l’offre à des segments plus ciblés du document. Le score final d’un CV est alors défini comme le score maximum obtenu parmi tous ses blocs :

$$\text{score}(CV) = \max_{i=1}^n \cos\left(\text{emb}(\text{Offre}), \text{emb}(b_i)\right) \quad (4)$$

où n est le nombre de blocs du CV et b_i le i -ème bloc.

Cela garantit que même une correspondance partielle mais forte avec l’offre est correctement détectée.

Cette approche permet ainsi de dépasser les limites des méthodes classiques, en prenant en compte le sens des mots et des expressions, et constitue le cœur du système de matching développé, qui sera présenté en détail dans le chapitre suivant.

2 Chapitre 2 : Extraction et nettoyage des CV au format PDF

Dans le cadre de notre projet, les CV constituent la principale source de données à exploiter. Cependant, ces documents sont majoritairement fournis au format PDF, un format conçu pour l’affichage visuel et non pour un traitement automatique.

Ainsi, avant toute analyse, il est nécessaire de transformer ces documents en un texte structuré et exploitable. Cette étape est cruciale, car la qualité des traitements ultérieurs, notamment le *chunking* et la vectorisation, dépend directement de la qualité de l’extraction.

Ce chapitre présente les différentes approches envisagées, les choix techniques retenus, ainsi que la méthode proposée pour extraire et nettoyer efficacement le contenu des CV.

2.1 Problématique

Les CV au format PDF ne sont pas directement exploitables pour une analyse automatique. Contrairement à un texte brut, un PDF contient des éléments positionnés dans l’espace, sans structure logique explicite.

De plus, les CV présentent souvent des mises en page complexes :

- organisation en plusieurs colonnes,
- présence d’icônes ou de symboles,
- utilisation de barres de progression pour représenter des compétences.

L’objectif de cette étape est donc d’extraire un texte propre et structuré à partir de ces documents, afin de préparer les étapes suivantes.

2.2 Choix techniques

Plusieurs bibliothèques Python permettent d’extraire du texte à partir de fichiers PDF, notamment PyPDF, pdfplumber et PyMuPDF, chacune ayant ses spécificités.

PyPDF permet une extraction simple du texte brut, mais ne fournit pas d’informations sur la structure ou la position des éléments, ce qui limite son utilisation pour des documents complexes comme les CV. [9]

pdfplumber offre une extraction plus précise avec accès aux coordonnées des éléments. Elle est particulièrement adaptée aux documents contenant des tableaux. Cependant, elle fournit des données très fines (mots ou caractères), nécessitant un traitement supplémentaire par la suite. [10]

PyMuPDF propose un bon compromis. Elle permet d’extraire efficacement le texte tout en fournissant une structure organisée (blocs, lignes, segments) ainsi que les coordonnées. [11]

De plus, PyMuPDF est réputée pour ses performances élevées. Des retours d’expérience issus de la communauté Python [12] indiquent également que PyMuPDF peut être plus rapide que d’autres bibliothèques dans certains cas d’usage, ce qui est un avantage important pour traiter un grand nombre de CV.

Ainsi, PyMuPDF apparaît comme la solution la plus adaptée à notre projet, en offrant un bon équilibre entre précision, simplicité d’utilisation, performance et capacité à gérer des cas

complexes comme les CV en colonnes.

2.3 Méthode proposée

La méthode mise en place se déroule en plusieurs étapes successives, permettant de passer d'un document PDF brut à un texte exploitable.

1. **Extraction des lignes** : le contenu du PDF est d'abord extrait sous forme de lignes de texte à l'aide de PyMuPDF. Pour chaque ligne, les coordonnées (positions horizontales et verticales) sont récupérées, ce qui permet de conserver des informations sur la structure visuelle du document.
2. **Détection des colonnes** : les lignes extraites sont ensuite regroupées en colonnes en fonction de leur position horizontale. Un clustering simple basé sur la proximité des coordonnées en abscisse est utilisé afin d'identifier les différentes zones du document, notamment dans le cas de CV en plusieurs colonnes.
3. **Reconstruction du texte** : une fois les colonnes identifiées, les lignes sont triées selon leur position verticale afin de reconstruire un ordre de lecture cohérent. L'analyse des écarts verticaux entre les lignes permet également de distinguer les retours à la ligne et les séparations de sections.
4. **Nettoyage** : enfin, un traitement de nettoyage est appliqué au texte reconstruit. Cette étape inclut la suppression des symboles inutiles (icônes, emojis), la normalisation des espaces, ainsi que la transformation de certains éléments visuels (comme les barres de progression) en informations textuelles exploitables.

La figure 6 présente le pipeline général de l'extraction et du nettoyage des CV, depuis le fichier PDF initial jusqu'au texte structuré exploitable par les étapes suivantes.

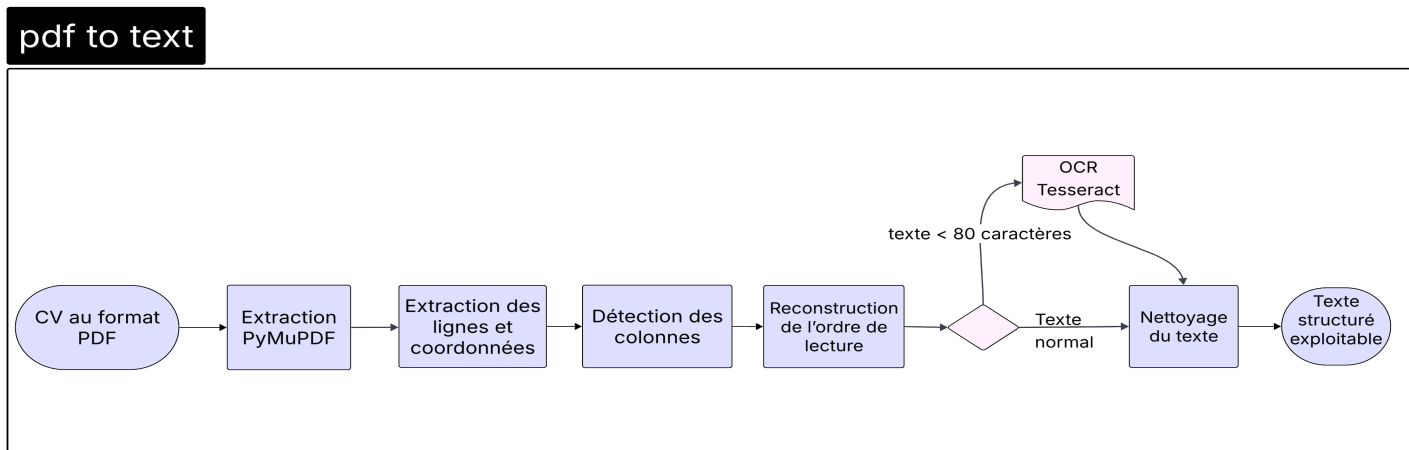


FIGURE 6 – Pipeline général d'extraction et de nettoyage des CV [14].

La figure 7 illustre l'intérêt de la reconstruction : contrairement à une extraction brute, notre méthode cherche à conserver un ordre de lecture plus cohérent, notamment pour les CV organisés en colonnes.



Nada Abir BOUHADJAR

Étudiante en troisième année licence informatique à la recherche d'une alternance pour septembre 2025.

Étudiante en troisième année licence informatique à la recherche d'une alternance pour septembre 2025.

Diplômes et Formations

- **Master en Intelligence Artificielle et Science des Données**
Depuis septembre 2025 Université de Montpellier Montpellier
- **Licence informatique**
Depuis 2024 Université de Pau et des pays de l'Adour France
Étudiante en troisième année licence informatique. 2024 - 2025
- **Licence professionnelle en ingénierie des systèmes informatiques et logiciels**
De 2023 à 2024
Université des sciences et de la technologie Houari-Boumediene (USTHB) Alger, Algérie
Obtention d'une licence professionnelle en ingénierie des systèmes informatiques et logiciels. 2023 - 2024
- **Baccalauréat scientifique**
De 2020 à 2021 Lycée Oussama Ibn Zaid Skikda, Algérie
Obtention d'un baccalauréat scientifique mention très bien. 2020 - 2021

Projets

- **Classification de textes avec NLP**
2025 Université de Montpellier Montpellier
Mise en place d'un pipeline de traitement du langage naturel : nettoyage du texte, vectorisation TF-IDF, entraînement de modèles de classification et analyse des erreurs.
- **Projet de prédiction de données**
2024 Université de Montpellier Montpellier
Construction d'un modèle de machine learning pour prédire une variable cible à partir de données structurées. Nettoyage des données, entraînement, évaluation avec des métriques adaptées et visualisation des résultats.
- **Tableau de bord de data visualisation**
2023 Université de Pau Pau
Création d'un tableau de bord interactif pour suivre des indicateurs clés, comparer des tendances et faciliter la prise de décision.

(a) CV original

Langues
Français
avancé.
Anglais
avancé (écrit et oral).
Arabe
maternelle.
Compétences
Informatique
Python, Java, SQL, R • Machine Learning, Deep Learning • NLP, classification, clustering • Pandas, NumPy, Scikit-learn • TensorFlow, PyTorch, Keras • Power BI, Matplotlib, Seaborn • Git, Linux, Jupyter Notebook • Bases de données relationnelles
Centres d'intérêt
Natation
Volley Ball
Rollers
Jeux de société
Nada Abir BOUHADJAR
Étudiante en troisième année licence informatique à la recherche d'une alternance pour septembre 2025.
Étudiante en troisième année licence informatique à la recherche d'une alternance pour septembre 2025.
Diplômes et Formations
Projets
bouhadjar.abir@gmail.com

(b) Extraction brute

Nada Abir BOUHADJAR
Étudiante en troisième année licence informatique à la recherche d'une alternance pour septembre 2025.
Étudiante en troisième année licence informatique à la recherche d'une alternance pour septembre 2025.

Diplômes et Formations
Master en Intelligence Artificielle et Science des Données
Depuis septembre 2025 Université de Montpellier Montpellier
Licence informatique
Depuis 2024 Université de Pau et des pays de l'Adour France
Étudiante en troisième année licence informatique. 2024 - 2025
Licence professionnelle en ingénierie des systèmes informatiques et logiciels
De 2023 à 2024
Université des sciences et de la technologie Houari-Boumediene (USTHB) Alger, Algérie
Obtention d'une licence professionnelle en ingénierie des systèmes informatiques et logiciels. 2023 - 2024
Baccalauréat scientifique
De 2020 à 2021 Lycée Oussama Ibn Zaid Skikda, Algérie
Obtention d'un baccalauréat scientifique mention très bien. 2020 - 2021

Projets
Classification de textes avec NLP
2025 Université de Montpellier Montpellier
Mise en place d'un pipeline de traitement du langage naturel : nettoyage du texte, vectorisation TF-IDF, entraînement de modèles de classification et analyse des erreurs.

(c) Extraction avec notre méthode

FIGURE 7 – Comparaison des résultats d'extraction entre l'approche brute et notre méthode.

Nous pouvons observer que l'extraction brute récupère bien le texte du CV, mais qu'elle ne respecte pas la structure visuelle du document. Dans le cas d'un CV organisé en plusieurs colonnes, les informations peuvent être mélangées ou lues dans un ordre incohérent : par exemple, des éléments de la colonne latérale peuvent être extraits avant ou au milieu du contenu principal. Le texte obtenu devient alors moins lisible et moins exploitable pour les étapes suivantes.

Notre méthode cherche au contraire à reconstruire un ordre de lecture plus naturel, ce qui permet de conserver une structure plus proche de celle du CV original. Le résultat obtenu est

donc plus clair, mieux structuré et plus adapté au traitement automatique.

Ainsi, cette étape améliore la qualité du texte transmis au système de matching. Un texte mal ordonné peut perturber la représentation sémantique du CV, tandis qu'un texte reconstruit de manière cohérente permet au modèle de mieux identifier les formations, les compétences, les expériences et les projets du candidat.

2.4 Gestion des cas particuliers

Certains fichiers PDF ne contiennent pas de texte exploitable (par exemple des documents scannés), car leur contenu est en réalité stocké sous forme d'image. Dans ce cas, une étape d'OCR (reconnaissance optique de caractères) est utilisée avec l'outil Tesseract afin de récupérer le texte à partir de l'image.

Cette approche permet de traiter des documents non textuels, mais elle présente certaines limites : la qualité du texte extrait dépend fortement de la qualité de l'image et peut contenir des erreurs.

2.5 Limites

Malgré ces améliorations, certaines limites subsistent :

- les CV avec des mises en page très complexes ou atypiques peuvent être mal reconstruits,
- les tableaux ne sont pas toujours correctement interprétés,
- l'OCR peut introduire des erreurs sur des documents de faible qualité.

Ces limites montrent que la méthode reste dépendante de la qualité et de la structure du document, et qu'elle pourrait être améliorée par des approches plus avancées de compréhension de documents.

2.6 Conclusion

Dans ce chapitre, nous avons présenté les défis liés à l'extraction de texte à partir de CV au format PDF ainsi que les différentes solutions existantes. Le choix de la bibliothèque PyMuPDF, combiné à des heuristiques basées sur les coordonnées, permet d'obtenir un texte structuré et exploitable dans la majorité des cas.

Malgré certaines limites, cette étape fournit une base suffisamment fiable pour les traitements suivants. Le texte nettoyé et restructuré pourra ensuite être découpé en blocs, puis transformé en embeddings afin de comparer automatiquement les CV aux offres d'emploi.

3 Chapitre 3 : Méthodologie de comparaison automatique CVs / Offres d'emploi

Face à un volume croissant de candidatures, identifier rapidement les profils les plus pertinents devient une tâche difficile à réaliser manuellement. L'objectif de ce chapitre est de présenter la méthodologie mise en place pour comparer automatiquement des CVs à des offres d'emploi : étant donné un ensemble de CVs et une offre, comment classer automatiquement les candidats par ordre de pertinence ?

L'approche retenue repose sur la représentation vectorielle des textes par embeddings et le calcul de similarité cosinus. Si l'on représente CVs et offres sous forme de vecteurs dans un espace sémantique, les candidats les plus pertinents seront ceux dont les vecteurs sont géométriquement les plus proches de l'offre.

Ce chapitre est organisé de la manière suivante. La Section 3.1 présente le modèle d'embeddings retenu. La Section 3.2 introduit le problème de la limite de taille et la notion de chunking. La Section 3.3 décrit les différentes stratégies de chunking explorées. La Section 3.4 présente le protocole expérimental, incluant le jeu de données et les métriques d'évaluation. La Section 3.5 analyse et discute les résultats obtenus. Enfin, la Section 3.6 conclut le chapitre.

3.1 Modèle d'embeddings retenu

Pour transformer les textes en vecteurs, nous avons utilisé le modèle paraphrase-multilingual-MiniLM-L12-v2 de Sentence-Transformers, disponible sur HuggingFace. Ce choix s'explique par la diversité linguistique du corpus : certains CVs sont rédigés en français, d'autres en anglais, certains mélangent les deux. Un modèle monolingue anglais comme all-MiniLM-L6-v2 pénaliserait mécaniquement les CVs en français face à une offre française.

À l'inverse, le modèle multilingue permet de projeter des textes issus de différentes langues dans un espace vectoriel commun. Cela rend possible une comparaison sémantique cohérente et équitable entre les CVs et les offres d'emploi, indépendamment de la langue utilisée.

3.2 Le problème de la taille des entrées

Les modèles d'embeddings présentent une limite de taille d'entrée, fixée à 128 tokens [13] pour MiniLM. Or, un CV dépasse souvent largement cette limite.

Cela pose deux problèmes : d'une part, le texte est tronqué, entraînant une perte d'informations importantes ; d'autre part, l'embedding obtenu peut diluer l'information pertinente en mélangeant des éléments essentiels avec des contenus moins utiles, ce qui réduit la précision de la représentation des compétences.

La solution retenue consiste à appliquer une stratégie de chunking, c'est-à-dire à découper chaque CV en blocs de taille réduite. Chaque bloc est ensuite encodé séparément, puis comparé à l'offre d'emploi. Le score global du CV est alors déterminé à partir du bloc présentant la plus forte similarité avec l'offre, comme le montre la figure ci-dessous.

Matching CV–Offre simplifié

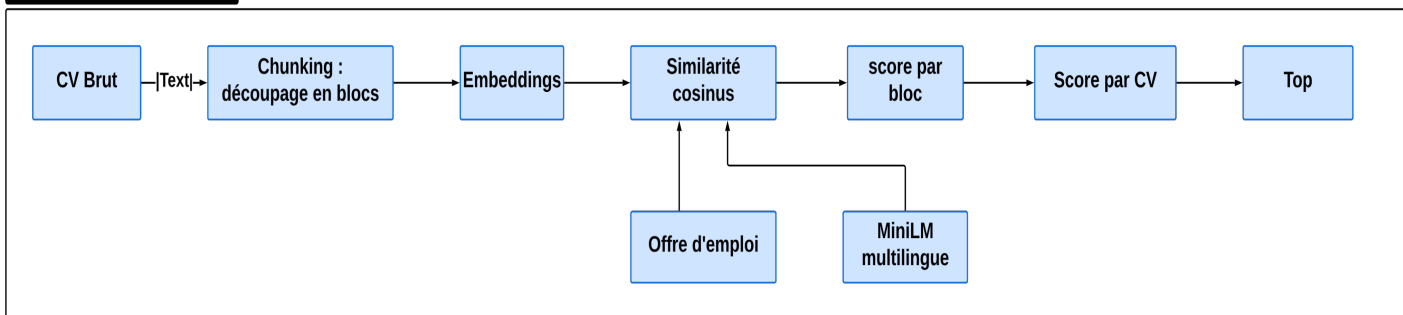


FIGURE 8 – Pipeline matching CV/offre (créé avec lucid.app) [14]

Cette approche permet de mieux préserver l’information sémantique en évitant les effets de troncature et de dilution, et garantit ainsi une représentation plus fidèle des différentes parties du CV.

3.3 Stratégies de chunking explorées

Six stratégies ont été implémentées et comparées. Voici leur principe de fonctionnement.

3.3.1 Découpage par sauts de ligne (chunk_by_newlines)

Cette stratégie consiste à découper le CV en segments en utilisant les sauts de ligne (`\n\n`) comme séparateurs. Chaque bloc correspond ainsi à une ligne ou un paragraphe du document original. Le schéma ci-dessous illustre ce processus.

Chunk by newlines

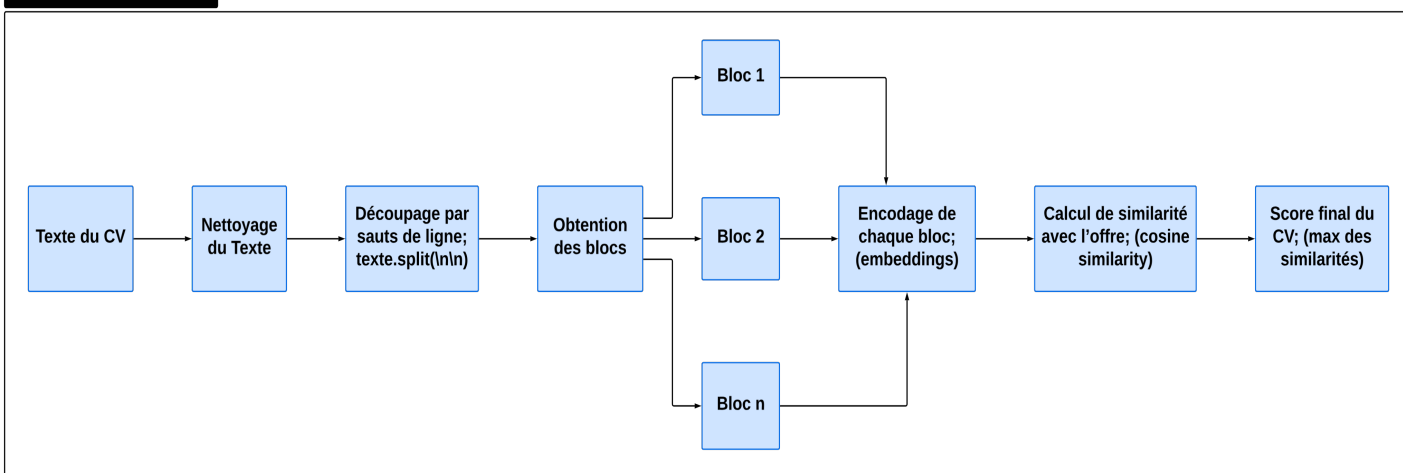


FIGURE 9 – Pipeline du découpage par sauts de ligne (chunk_by_newlines) [14]

3.3.1.1 Avantages et inconvénients

Avantages	Inconvénients
Méthode simple et rapide à implémenter	Taille des blocs non contrôlée
Faible coût de calcul	Génère souvent des blocs trop courts ou trop longs
Facilement interprétable	Perte de contexte entre les lignes
Permet un premier niveau de segmentation	Sensible au format du CV (PDF, copier-coller, etc.)
	Peut séparer des informations liées (ex : titre + description)

TABLE 1 – Avantages et inconvénients du découpage par sauts de ligne

Cette méthode constitue une approche simple et intuitive du chunking, mais ses limites en termes de cohérence sémantique et de contrôle de la taille des segments motivent l’exploration de stratégies plus avancées.

3.3.1.2 Exemple expérimental

Lors des expérimentations, cette stratégie a montré certaines limites. Par exemple, pour un CV contenant les informations suivantes :

EXPÉRIENCE

ML Engineer, Airbus AI Toulouse, 2020 à aujourd’hui

\n

- Développement de modèles de classification et de régression pour la maintenance prédictive
- Depuis 2022 : travaux sur le NLP pour l’analyse de rapports techniques

Le découpage par sauts de ligne produit :

[EXPÉRIENCE

ML Engineer, Airbus AI Toulouse, 2020 à aujourd’hui]

[- Développement de modèles de classification et de régression pour la maintenance prédictive

- Depuis 2022 : travaux sur le NLP pour l’analyse de rapports techniques]

Analyse

Le découpage par sauts de ligne conserve des blocs globalement cohérents, mais peut séparer des informations fortement liées, comme le poste et les missions associées. Cette fragmentation peut entraîner une perte de contexte sémantique et réduire la pertinence du score de similarité.

Simple à mettre en œuvre et facilement interprétable, cette stratégie constitue un bon point de départ. La section suivante explore une approche permettant de mieux contrôler la taille des blocs générés.

3.3.2 Découpage par mots fixes avec chevauchement (chunk_by_words)

Cette stratégie consiste à découper le texte en segments de taille fixe, définie en nombre de mots. Afin de préserver le contexte entre les segments, un chevauchement (overlap) est introduit entre les blocs consécutifs. Chaque segment partage ainsi une partie de son contenu avec le précédent.

Voici un schéma montrant un exemple d'un CV contenant 200 mots, avec `max_words=80` et `overlap=20`.

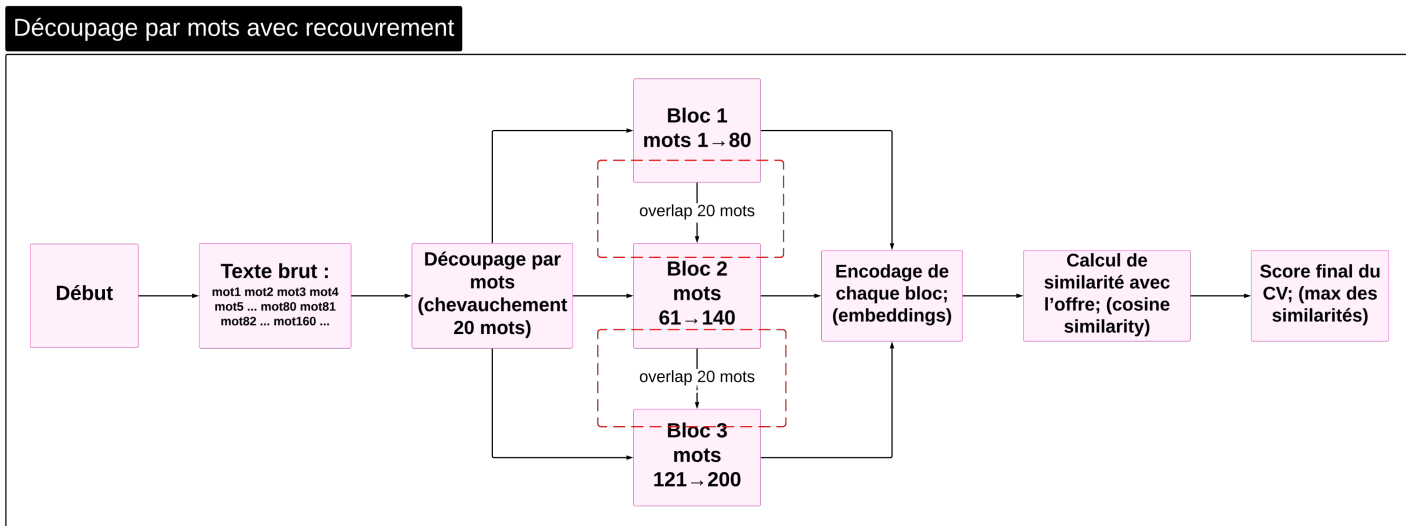


FIGURE 10 – Pipeline du découpage par mots fixes avec chevauchement [14]

3.3.2.1 Avantages et inconvénients

Avantages	Inconvénients
Contrôle précis de la taille des blocs	Découpage non aligné avec la structure du texte
Réduction du risque de dépassement de tokens	Peut couper des phrases en plein milieu
Le chevauchement préserve partiellement le contexte	Redondance d'information entre blocs
	Moins interprétable
	Augmentation du coût de calcul

TABLE 2 – Avantages et inconvénients du découpage par mots fixes avec chevauchement

3.3.2.2 Exemple expérimental

Lors des expérimentations, cette stratégie a montré certaines limites. Pour un CV contenant :

[ML Engineer chez Airbus AI. Développement de modèles de classification.
Travaux récents en NLP pour l'analyse de rapports techniques.]

Avec un découpage en fenêtres de 10 mots et un chevauchement de 3 mots :

[ML Engineer chez Airbus AI. Développement de modèles de classification.]
 [modèles de classification. Travaux récents en NLP pour l'analyse]
 [NLP pour l'analyse de rapports techniques.]

Analyse

Le chevauchement permet de conserver une partie du contexte entre les blocs, limitant ainsi la perte d'information. Cependant, le découpage reste arbitraire et peut fragmenter des unités sémantiques naturelles, ce qui peut affecter la qualité des embeddings.

Le contrôle de la taille et le chevauchement constituent des avantages réels par rapport au découpage par sauts de ligne. Toutefois, l'absence d'alignement avec la structure linguistique du texte motive l'exploration d'une approche plus naturelle, présentée dans la section suivante.

3.3.3 Découpage par phrases (chunk_by_sentences)

Cette stratégie consiste à segmenter le texte en phrases, en utilisant la ponctuation (points, points d'interrogation, points d'exclamation, etc.) comme délimiteurs. Chaque phrase est ensuite considérée comme une unité sémantique indépendante à encoder. Le schéma ci-dessous illustre ce processus.

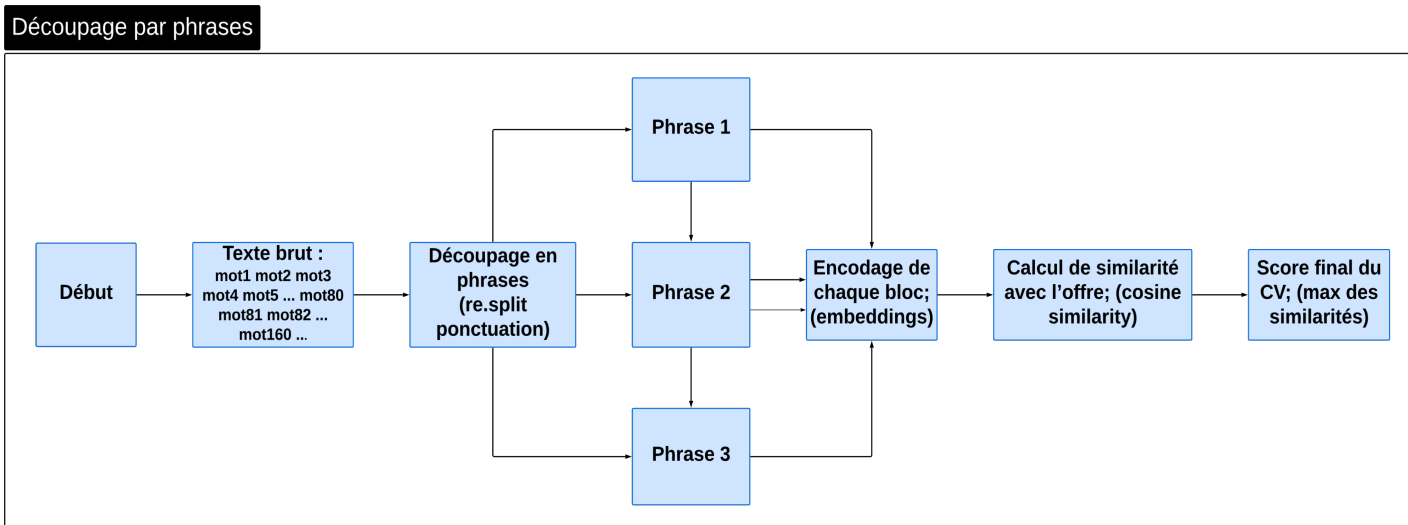


FIGURE 11 – Pipeline du découpage par phrases [14]

3.3.3.1 Avantages et inconvénients

Avantages	Inconvénients
Segmentation plus naturelle que par mots	Phrases parfois trop courtes
Respect des unités sémantiques	Perte de contexte entre phrases
Bonne interprétabilité	Sensible à la ponctuation (CVs mal formatés)
Facile à implémenter	Peut isoler des phrases peu informatives
	Taille des blocs non contrôlée

TABLE 3 – Avantages et inconvénients du découpage par phrases

3.3.3.2 Exemple expérimental

Considérons le texte suivant :

[ML Engineer chez Airbus AI. Développement de modèles de classification pour la maintenance prédictive. Travaux récents en NLP pour l'analyse de rapports techniques.]

Après découpage en phrases :

[ML Engineer chez Airbus AI.]

[Développement de modèles de classification pour la maintenance prédictive.]

[Travaux récents en NLP pour l'analyse de rapports techniques.]

Analyse

Le découpage par phrases produit des segments sémantiquement cohérents, chaque phrase constituant une unité de sens complète. Cependant, sur des CVs mal ponctués ou rédigés sous forme de listes sans points, cette stratégie génère peu de coupures et produit des blocs trop longs, ce qui peut dépasser la limite de tokens du modèle.

Le découpage par phrases améliore la cohérence sémantique des blocs par rapport aux approches précédentes. Toutefois, sa dépendance à la ponctuation le rend fragile sur des CVs hétérogènes. La section suivante explore une approche qui s'affranchit de ces contraintes en basant les coupures directement sur le sens du texte.

3.3.4 Chunking sémantique (chunk_semantic)

Cette stratégie repose sur une approche sémantique du découpage. Le texte est d'abord segmenté en phrases, puis la similarité cosinus est calculée entre chaque paire de phrases consécutives. Une coupure est introduite lorsque la similarité chute en dessous d'un seuil, déterminé de manière adaptative pour chaque document. Le schéma ci-dessous illustre ce processus.

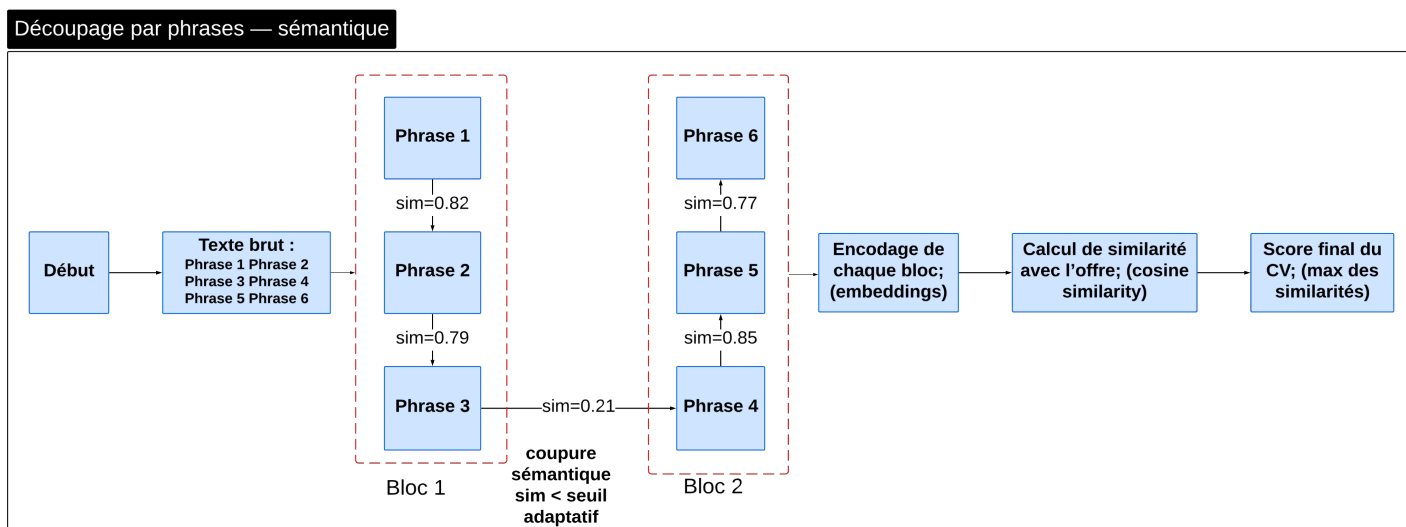


FIGURE 12 – Pipeline du découpage sémantique [14]

3.3.4.1 Avantages et inconvénients

Avantages	Inconvénients
Découpage basé sur le sens du texte	Dépend fortement du seuil choisi
Adaptatif au contenu du document	Sensible aux variations faibles de similarité
Segments plus cohérents sémantiquement	Moins efficace sur textes homogènes
Méthode plus « intelligente » que les heuristiques simples	Coût de calcul plus élevé
	Implémentation plus complexe

TABLE 4 – Avantages et inconvénients du chunking sémantique

3.3.4.2 Exemple expérimental

Considérons le texte suivant :

ML Engineer chez Airbus AI. Développement de modèles de classification.
Travaux en NLP pour l'analyse de rapports techniques.
Passionné de photographie et de sport.

Similarités entre phrases :

$\text{sim}(1, 2) = \text{élevée} \rightarrow \text{même thème (ML)}$
 $\text{sim}(2, 3) = \text{faible} \rightarrow \text{changement de thème}$

Découpage obtenu :

[ML Engineer chez Airbus AI. Développement de modèles de classification.
Travaux en NLP pour l'analyse de rapports techniques.]
[Passionné de photographie et de sport.]

Analyse

Le chunking sémantique permet d'identifier automatiquement les ruptures thématiques du texte, sans dépendre de la ponctuation ni de la structure visuelle du CV. Sur l'exemple présenté, la séparation entre les informations professionnelles et les centres d'intérêt est correctement détectée. Cependant, sur des CVs homogènes, par exemple entièrement techniques, les similarités entre phrases consécutives restent élevées, rendant le seuil peu discriminant et produisant peu de coupures.

Cette stratégie est la plus adaptative des approches explorées jusqu'ici, car elle s'appuie sur le contenu sémantique plutôt que sur des règles fixes. Néanmoins, aucune stratégie prise isolément ne s'avère optimale sur tous les types de CVs. La section suivante propose une approche de fusion qui combine les scores de plusieurs stratégies afin d'obtenir un classement plus robuste.

3.3.5 Fusion des stratégies de chunking (chunk_fusion)

Afin de tirer parti des différentes stratégies de chunking explorées, une approche de fusion a été mise en place. L'idée est de ne pas dépendre d'une seule méthode de découpage, mais de combiner leurs résultats afin d'obtenir un classement plus robuste et fiable des candidats.

Chaque stratégie produit un score de similarité pour chaque CV. Ces scores sont ensuite agrégés à l'aide d'une pondération automatique, permettant de valoriser les stratégies les plus discriminantes. Le schéma ci-dessous illustre ce processus

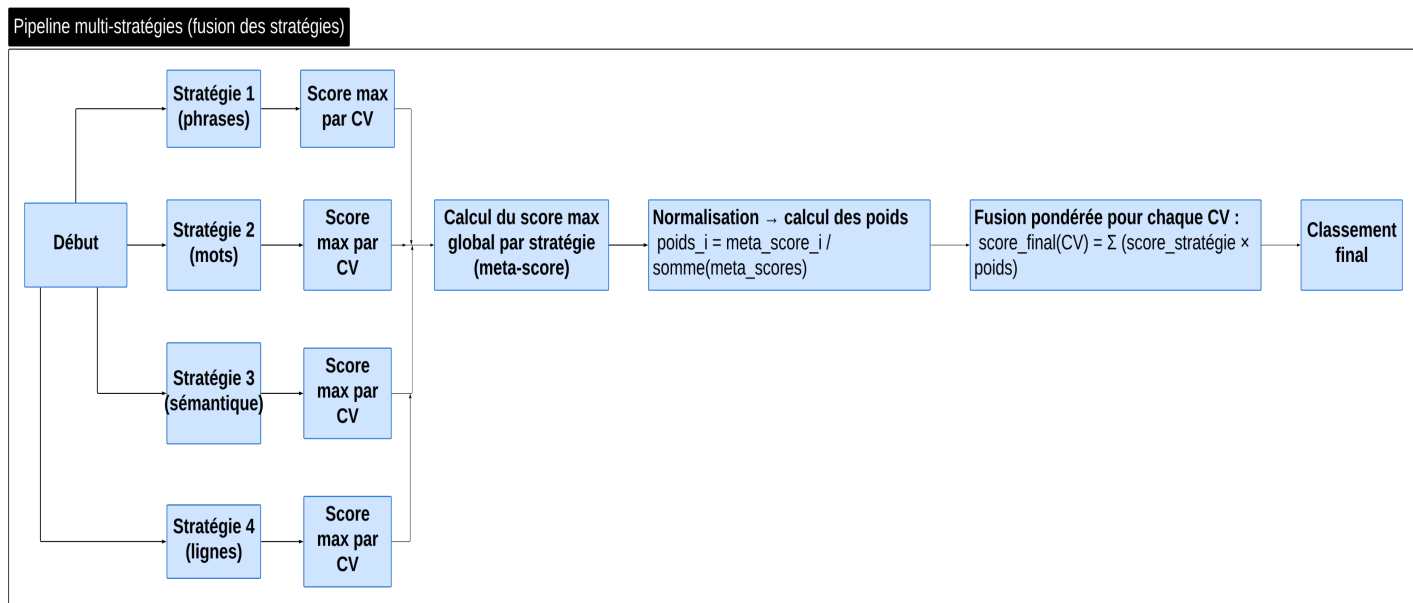


FIGURE 13 – Pipeline de la fusion des stratégies de chunking [14]

3.3.5.1 Avantages et inconvénients

Avantages	Inconvénients
Combine les forces de plusieurs stratégies	Dépend de la qualité des stratégies individuelles
Réduction du risque d'erreur d'une seule méthode	Pondération heuristique (non apprise)
Améliore la robustesse du classement	Peut lisser les différences entre candidats
Adaptatif (poids automatiques)	Coût de calcul plus élevé
Meilleure généralisation sur différents types de CV	Plus difficile à interpréter

TABLE 5 – Avantages et inconvénients de la fusion des stratégies de chunking

3.3.5.2 Exemple expérimental

Considérons trois stratégies donnant les scores suivants pour un CV :

Stratégie	Score	Poids
chunk_by_words	0.82	0.30
chunk_by_sentences	0.65	0.20
chunk_semantic	0.78	0.28
chunk_by_newlines	0.50	0.22

TABLE 6 – Scores par stratégie et poids associés

Le score final fusionné est calculé comme suit :

$$\text{score} = 0.82 \times 0.30 + 0.65 \times 0.20 + 0.78 \times 0.28 + 0.50 \times 0.22 \approx 0.71 \quad (5)$$

Analyse

La fusion permet de compenser les faiblesses de chaque stratégie en s'appuyant sur les plus performantes. Sur nos expérimentations avec 20 CVs aux formats variés, la fusion pondérée a produit un classement cohérent avec le jugement humain, là où certaines stratégies prises isolément donnaient des résultats instables.

La robustesse de cette approche vient du fait que les poids sont calculés automatiquement à partir des performances de chaque stratégie sur le corpus évalué, les stratégies les plus discriminantes contribuent davantage au score final. Néanmoins, ces poids restent heuristiques et non appris, ce qui constitue une limite dans un contexte de généralisation à de nouveaux corpus. La section suivante propose une approche plus intégrée, combinant découpage structurel et fusion sémantique au sein d'un même pipeline adaptatif.

3.3.6 Chunking hiérarchique adaptatif (chunk_adaptatif)

Afin de dépasser les limites des stratégies précédentes, on propose une approche de chunking hiérarchique adaptatif. Cette méthode repose sur un découpage progressif du texte selon plusieurs niveaux structurels, combiné à une fusion sémantique des segments.

L'objectif est de produire des blocs à la fois cohérents sémantiquement et contraints en taille (max_words), tout en s'adaptant automatiquement à la structure du CV. Les figures 14 et 15 illustrent respectivement le pipeline global de la méthode et le détail du découpage hiérarchique, tandis que la figure 16 présente le mécanisme de fusion sémantique des blocs.

Schéma global de la méthode

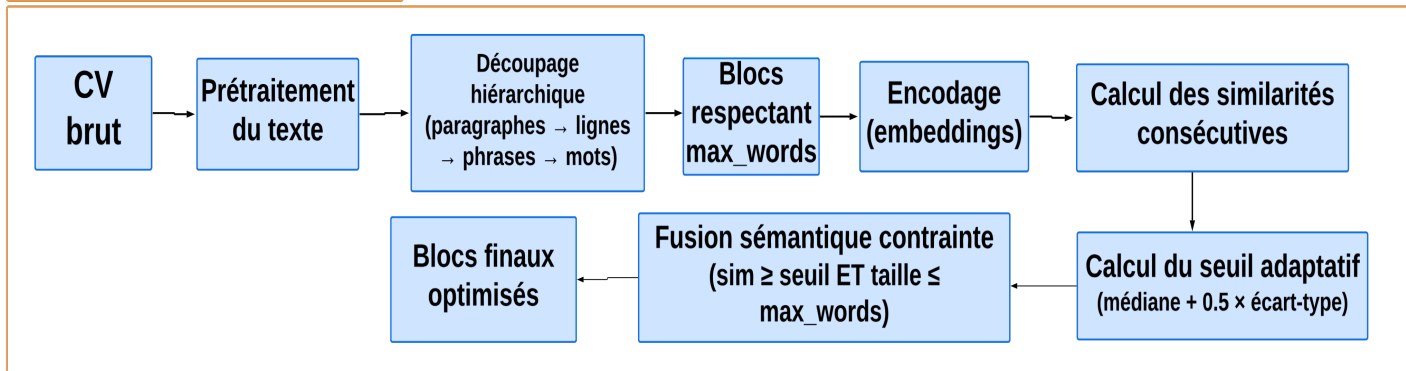


FIGURE 14 – Pipeline global de la méthode de découpage adaptatif [14]

Détail du découpage hiérarchique

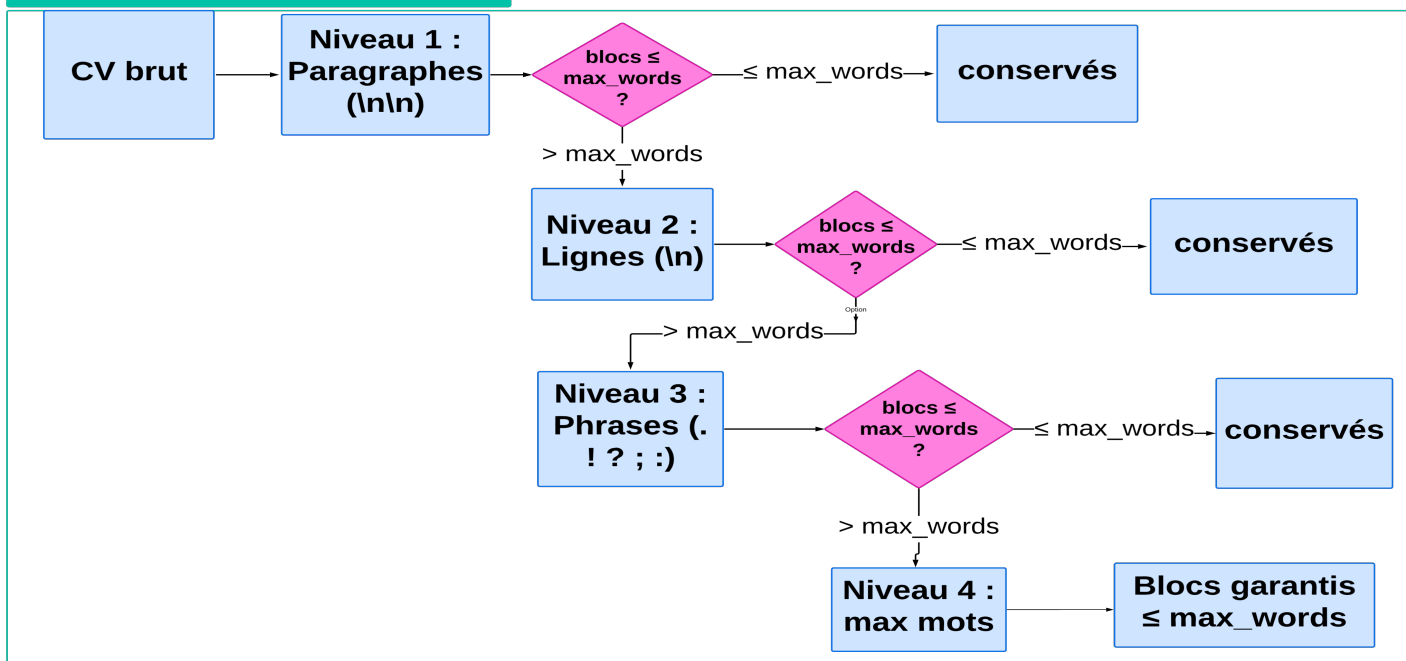


FIGURE 15 – Détails du découpage adaptatif [14]

Schéma de fusion sémantique

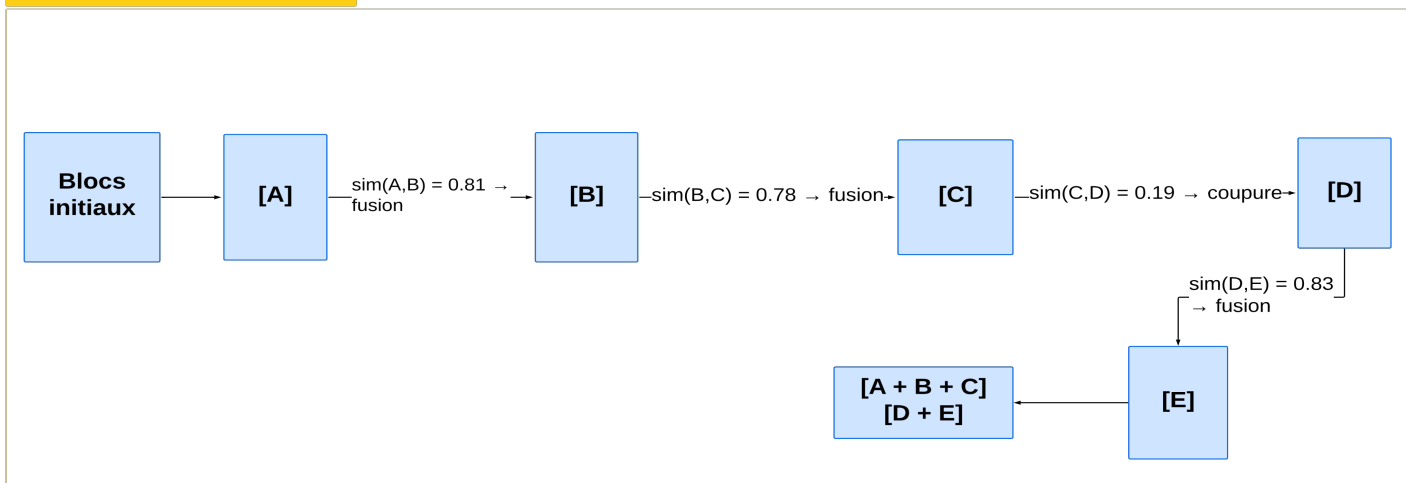


FIGURE 16 – Pipeline de la fusion sémantique des blocs [14]

3.3.6.1 Avantages et inconvénients

Avantages	Inconvénients
Découpage multi-niveaux robuste	Implémentation plus complexe
Adaptation automatique à la structure du CV	Coût de calcul plus élevé
Contrôle strict de la taille (<code>max_words</code>)	Dépend de la qualité des embeddings
Fusion sémantique intelligente	Plusieurs paramètres à régler
Bonne gestion des CVs hétérogènes	Moins facilement interprétable
Combine structure et sémantique	

TABLE 7 – Avantages et inconvénients du chunking hiérarchique adaptatif

3.3.6.2 Exemple expérimental

Considérons le texte suivant :

ML Engineer chez Airbus AI. Développement de modèles de classification.
Travaux récents en NLP pour l'analyse de rapports techniques.
Passionné de photographie et de sport. Disponible immédiatement.

Le découpage hiérarchique s'effectue en quatre niveaux successifs. Le texte est d'abord découpé par paragraphes (`\n\n`) ; si un bloc dépasse `max_words=80` mots, il descend au niveau suivant : lignes simples (`\n`), puis phrases (`., !, ?, ;`), et enfin mots en dernier recours. Une fois les blocs obtenus, un seuil adaptatif est calculé automatiquement pour chaque document :

$$\text{seuil} = \text{médiane}(\text{sims}) + 0,5 \times \text{écart-type}(\text{sims})$$

$$\text{seuil} = \max(0,2, \min(\text{seuil}, 0,85))$$

Les blocs dont la similarité dépasse ce seuil et dont la taille fusionnée reste inférieure à `max_words` sont regroupés. On obtient ainsi :

[ML Engineer chez Airbus AI. Développement de modèles de classification. Travaux récents en NLP pour l'analyse de rapports techniques.]

[Passionné de photographie et de sport. Disponible immédiatement.]

Analyse

La stratégie adaptative combine les avantages des approches structurelles et sémantiques. Le découpage hiérarchique garantit qu'aucun bloc ne dépasse la limite de tokens du modèle, tandis que la fusion sémantique regroupe intelligemment les segments thématiquement proches. Contrairement au chunking sémantique seul, le seuil est ici borné entre 0,2 et 0,85 pour éviter les comportements extrêmes (trop de fusions ou aucune).

Cette approche combine structure et sémantique de manière intégrée, ce qui la rend robuste même face à des CVs hétérogènes ou mal formatés. L'ensemble des six stratégies ayant été présentées, la section suivante décrit le jeu de données utilisé pour les évaluer expérimentalement.

3.4 Protocole d'évaluation

3.4.1 Jeu de données de test

Afin d'évaluer et de comparer objectivement les six stratégies de chunking, nous avons constitué un jeu de données de test composé de 20 CVs et d'une offre d'emploi cible. L'offre porte sur un poste d'ingénieur NLP, requérant des compétences en Python, HuggingFace, embeddings, similarité cosinus, FastAPI et Git.

Les 20 CVs ont été conçus pour couvrir un spectre large de profils, allant des candidats très pertinents aux profils sans lien avec le poste. On distingue quatre catégories :

- **Profils très pertinents** (CV1, CV2, CV3, CV10, CV15) : ingénieurs et chercheurs NLP avec une expérience directe en embeddings, similarité cosinus, FastAPI et Git.
- **Profils pertinents** (CV4, CV9, CV14) : profils NLP juniors ou MLOps avec une bonne maîtrise des outils, mais une expérience moindre.
- **Profils moyennement pertinents** (CV5, CV6, CV12) : profils data science ou backend ayant une exploration récente du NLP, sans expertise consolidée.
- **Profils peu ou non pertinents** (CV7, CV8, CV11, CV13, CV16, CV17, CV18, CV19, CV20) : profils DevOps, développeurs mobiles, cybersécurité ou chefs de projet sans lien avec le NLP.

3.4.2 Métriques d'évaluation

Pour évaluer la qualité du classement produit par chaque stratégie, nous comparons le classement obtenu automatiquement à un classement de référence établi manuellement, en tenant compte du profil de chaque candidat par rapport à l'offre. Cinq métriques complémentaires sont utilisées :

- **NDCG** (Normalized Discounted Cumulative Gain) : mesure la qualité globale du classement en pénalisant davantage les erreurs en tête de liste. Un NDCG de 1 indique un classement parfait.
- **Corrélation de Spearman** : mesure la corrélation entre les rangs obtenus et les rangs attendus. Une valeur proche de 1 indique un classement globalement cohérent avec la

référence.

- **Precision@3** et **Precision@5** : mesurent la proportion de candidats pertinents retrouvés parmi les 3 et 5 premiers résultats. Ces métriques reflètent le comportement réel d’un recruteur, qui consulte en priorité les premiers candidats du classement.
- **MRR** (Mean Reciprocal Rank) : mesure à quelle position apparaît le meilleur candidat attendu. Un MRR de 1 signifie que le meilleur candidat est classé premier.

Ces métriques sont complémentaires : le NDCG et Spearman évaluent le classement global, tandis que Precision@k et MRR se concentrent sur la pertinence en tête de liste, ce qui correspond à l’usage réel du système.

3.5 Résultats et discussion

Le tableau 8 présente les scores obtenus par chaque stratégie sur l’ensemble des cinq métriques.

Stratégie	NDCG	Spearman	P@3	P@5	MRR
chunk_by_newlines	0.9516	0.6827	0.67	0.60	1.00
chunk_by_words	0.9361	0.6647	0.33	0.40	1.00
chunk_by_sentences	0.8808	0.2947	0.00	0.20	0.10
chunk_semantic	0.9554	0.6030	0.67	0.60	1.00
chunk_adaptatif	0.9789	0.7805	1.00	0.60	1.00
chunk_fusion	0.9369	0.6241	0.33	0.40	1.00

TABLE 8 – Comparaison des six stratégies de chunking sur cinq métriques d’évaluation. Les meilleures valeurs par métrique sont indiquées en gras.

Plusieurs observations se dégagent de ces résultats.

Le chunking adaptatif domine sur la majorité des métriques. Il obtient le meilleur NDCG (0.9789), la meilleure corrélation de Spearman (0.7805) et une Precision@3 parfaite de 1.00, signifiant que les trois meilleurs candidats identifiés par le système correspondent exactement aux trois meilleurs candidats du classement de référence. Ce résultat confirme que combiner découpage structurel et fusion sémantique produit un classement globalement plus fidèle au jugement humain.

Le chunking par sauts de ligne reste compétitif malgré sa simplicité. Avec un NDCG de 0.9516 et le meilleur score de Precision@5 (0.60), il performe bien sur les CVs bien structurés de notre jeu de données. Cependant, cette performance est fragile : sur des CVs rédigés en texte continu sans sauts de ligne, comme CV2, CV8 ou CV13, la stratégie ne produit qu’un seul bloc, ce qui équivaut à un embedding global du document entier.

Le chunking par phrases est la stratégie la moins robuste. Avec une Precision@3 nulle et un MRR de seulement 0.10, il échoue à placer les bons candidats en tête de classement. Ce résultat s’explique par la nature des CVs du jeu de données : beaucoup sont rédigés sous forme de listes sans ponctuation explicite, ce qui entraîne un découpage en très peu de blocs trop longs, dépassant la limite de tokens du modèle.

La fusion pondérée ne surpasse pas le chunking adaptatif. Contrairement à l’intuition initiale, combiner les quatre stratégies par pondération automatique ne produit pas le meilleur classement. La fusion lisse les différences entre candidats et pénalise les stratégies fortes lorsqu’elles sont diluées par des stratégies moins performantes comme le chunking par phrases.

Le MRR révèle une robustesse partagée. Cinq stratégies sur six obtiennent un MRR de 1.00, ce qui signifie que le meilleur candidat — James Fletcher (CV3), Research Scientist NLP — est systématiquement classé premier, quelle que soit la stratégie utilisée. Seul le chunking par phrases échoue sur ce point ($\text{MRR} = 0.10$), confirmant ses difficultés structurelles.

3.6 Conclusion

Ce chapitre a présenté la méthodologie complète de comparaison automatique de CVs à une offre d'emploi, depuis le choix du modèle d'embeddings jusqu'à l'évaluation quantitative des stratégies de chunking.

L'exploration de six stratégies et leur évaluation sur un jeu de données de 20 CVs aux formats variés met en évidence un constat central : aucune stratégie n'est universellement optimale, et la qualité du chunking dépend fortement de la structure et du format des documents traités. Les résultats montrent néanmoins que le chunking hiérarchique adaptatif constitue la solution la plus robuste, en obtenant les meilleures performances sur quatre métriques sur cinq.

Ce constat rejoint une problématique centrale des systèmes RAG industriels : le chunking reste l'étape la plus délicate à optimiser. Le chapitre suivant présente l'interface web développée pour permettre l'utilisation concrète de ce système par un recruteur.

4 Chapitre 4 : Interface web et intégration

Afin de rendre accessible le système de matching à un recruteur, nous avons développé une interface web complète. Ce chapitre présente l'architecture de l'application, l'implémentation technique, les fonctionnalités offertes, ainsi que les résultats des tests utilisateurs.

Ce chapitre est structuré de la manière suivante : la Section 4.1 présente l'architecture générale de l'application web. La Section 4.2 détaille l'implémentation du backend avec Flask. La Section 4.3 décrit l'interface utilisateur et son comportement dynamique. La Section 4.4 expose les fonctionnalités avancées (fiche candidat, visualisation CV). Enfin, la Section 4.5 présente les tests et la validation.

4.1 Architecture générale de l'application

L'application web suit une architecture client-serveur classique, illustrée par la Figure 17. Le navigateur (client) communique avec un serveur Flask (backend) qui orchestre l'exécution du moteur IA. Cette séparation des responsabilités permet de maintenir une interface réactive et indépendante des calculs lourds.

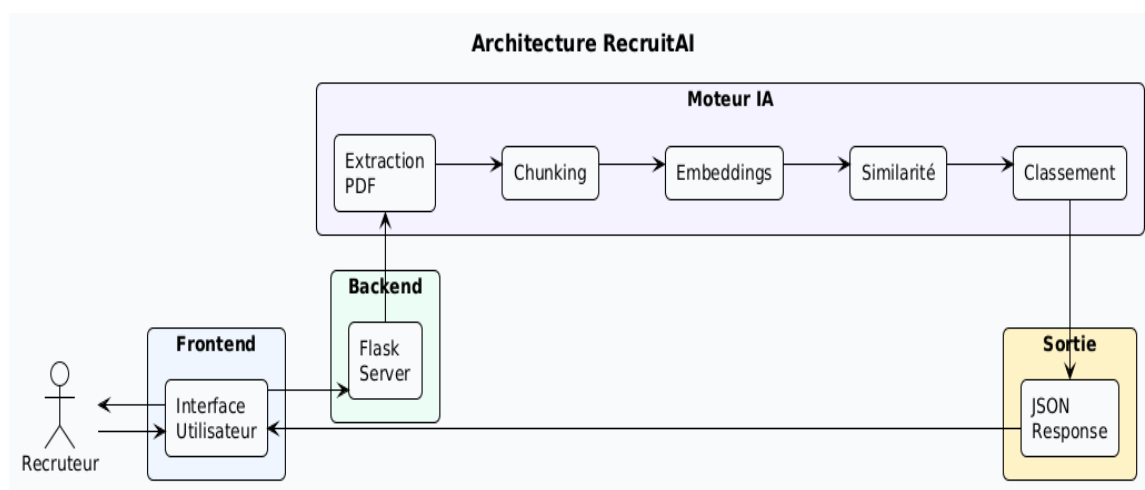


FIGURE 17 – Architecture générale de l'application web.

Le schéma met en évidence trois composants principaux :

- **Le navigateur** : il affiche l'interface utilisateur, gère les interactions (drag & drop, clics) et envoie les requêtes au serveur. Aucun calcul d'IA n'est réalisé côté client.
- **Le serveur Flask** : il reçoit les requêtes, extrait le texte des CVs, applique les stratégies de chunking, calcule les embeddings et retourne le classement au format JSON.
- **Le moteur IA** : il regroupe les modules d'extraction de texte, de chunking et de similarité cosinus. Il est intégré au serveur et n'est pas exposé directement au client.

Cette architecture permet de séparer clairement les préoccupations : le frontend gère l'interaction utilisateur et le rendu, tandis que le backend orchestre l'exécution du moteur IA. Le recruteur n'a ainsi aucune manipulation technique à effectuer ; il lui suffit de déposer ses fichiers et de lancer l'analyse.

4.2 Diagramme de cas d'utilisation

Afin de modéliser les interactions du recruteur avec le système, nous avons réalisé un diagramme de cas d'utilisation, présenté en Figure 18.

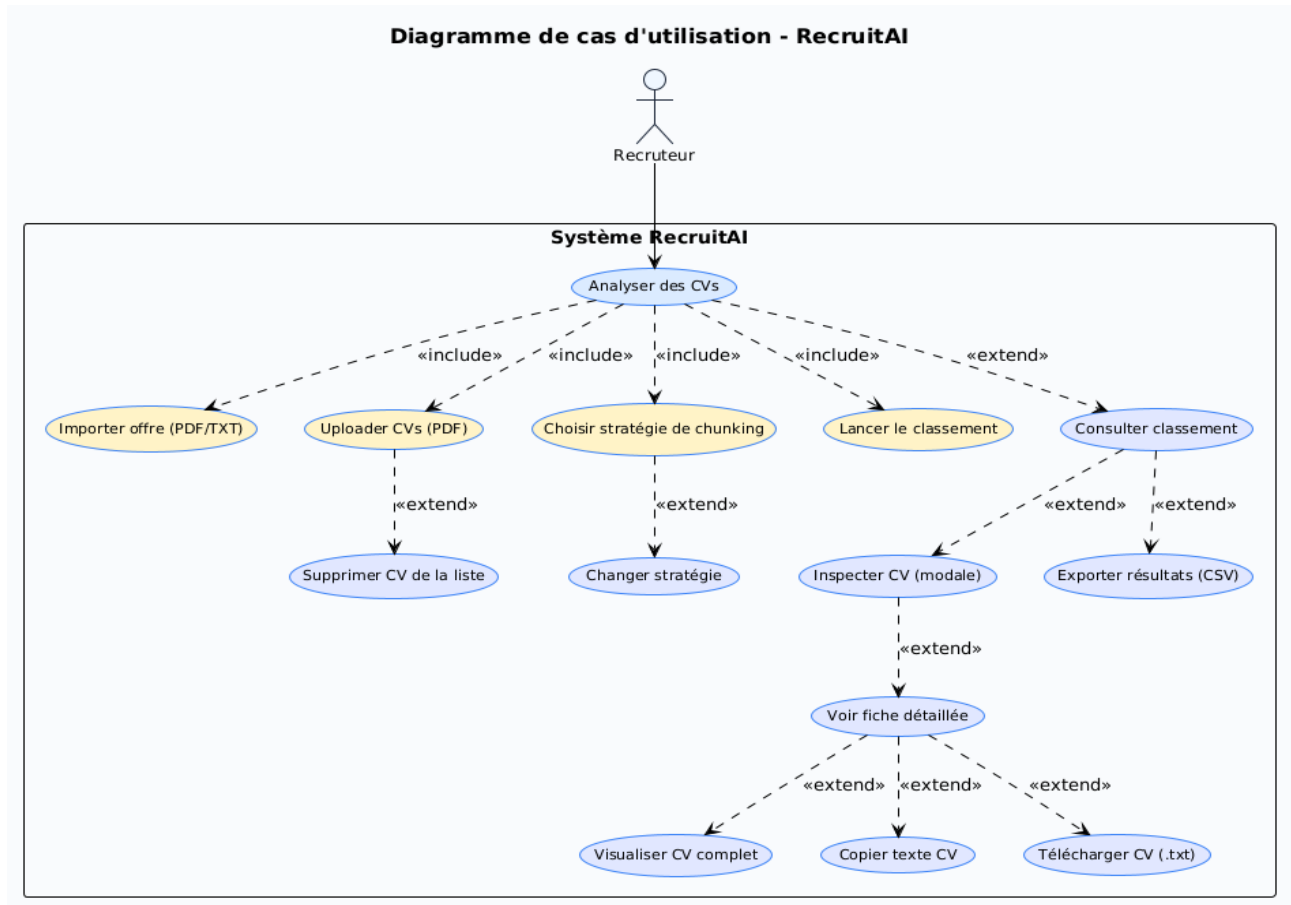


FIGURE 18 – Diagramme de cas d'utilisation – RecruitAI.

Le diagramme identifie les cas d'utilisation principaux suivants :

- **Analyser des CVs** : le recruteur dépose une offre d'emploi et des fichiers PDF, puis lance le traitement. Il s'agit du cas d'utilisation central.
- **Consulter le classement** : après analyse, le recruteur visualise la liste triée des candidats avec leurs scores globaux.
- **Inspecter un CV** : le recruteur peut ouvrir une fenêtre modale contenant le texte intégral du CV pour une vérification manuelle.
- **Consulter la fiche détaillée** : pour chaque candidat, une fiche synthétique présente les informations clés (email, téléphone, compétences détectées, bloc décisif).
- **Changer la stratégie de chunking** : le recruteur peut choisir parmi quatre stratégies (adaptatif, sauts de ligne, taille fixe, par phrases) avant de lancer l'analyse.
- **Exporter le CV en texte** : le recruteur peut télécharger le texte extrait d'un CV au format TXT.

Les relations d'extension (extend) indiquent que certaines actions (inspection du CV, consultation de la fiche détaillée, export) sont des variantes de la consultation du classement.

Le diagramme montre que le parcours principal du recruteur est centré sur l'analyse de CVs : il dépose une offre, importe les documents, lance le traitement, puis consulte les résultats. Les

autres actions, comme l'inspection d'un CV ou l'export du texte, viennent compléter ce parcours principal.

4.3 Implémentation du backend avec Flask

Le backend de l'application est développé avec le micro-framework Flask (Python). Ce choix s'explique par sa légèreté et sa facilité d'intégration avec les modules Python déjà développés pour l'extraction de texte et le calcul des similarités.

4.3.1 Architecture du serveur

Le serveur Flask est organisé en trois couches :

1. **Couche de présentation** : elle sert la page HTML statique (via la route `/`) et les ressources associées (CSS, JavaScript).
2. **Couche applicative** : elle contient l'endpoint `/upload` qui traite les requêtes d'analyse.
3. **Couche d'intégration** : elle appelle les fonctions du moteur IA (`extraire_texte_cv`, `classer_cvs`) et formate les résultats en JSON.

4.3.2 Endpoints exposés

Route principale – GET `/` : cette route renvoie le fichier `index.html` qui constitue l'interface utilisateur. Elle ne nécessite aucune authentification et sert de point d'entrée à l'application.

Endpoint d'analyse – POST `/upload` : il s'agit du cœur du backend. Il reçoit une requête au format `multipart/form-data` contenant deux champs :

- **offre** : une chaîne de caractères représentant l'offre d'emploi.
- **cvs** : une liste de fichiers PDF.

Le traitement s'effectue selon les étapes suivantes :

1. Récupération des données : Flask extrait le texte de l'offre et la liste des fichiers.
2. Sauvegarde temporaire : chaque fichier est enregistré dans le répertoire `uploads` avec un nom sécurisé (via `secure_filename`).
3. Extraction du texte : la fonction `extraire_texte_cv` est appelée pour chaque PDF.
4. Classement : la fonction `classer_cvs` applique la stratégie de chunking choisie et calcule les similarités.
5. Nettoyage : les fichiers temporaires sont supprimés.
6. Réponse JSON : le dictionnaire des résultats est sérialisé et renvoyé au client.

4.3.3 Gestion des fichiers uploadés

La gestion des fichiers est critique pour la sécurité et la robustesse. Les mesures suivantes ont été prises :

- **Extensions autorisées** : seuls les fichiers `.pdf` sont acceptés.
- **Nom sécurisé** : Flask fournit la fonction `secure_filename` qui élimine les caractères dangereux.

- **Nettoyage systématique** : les fichiers sont supprimés immédiatement après l'analyse, qu'elle ait réussi ou échoué.
- **Limite de taille** : la taille maximale d'un fichier est fixée à 50 Mo.

4.3.4 Intégration du moteur IA

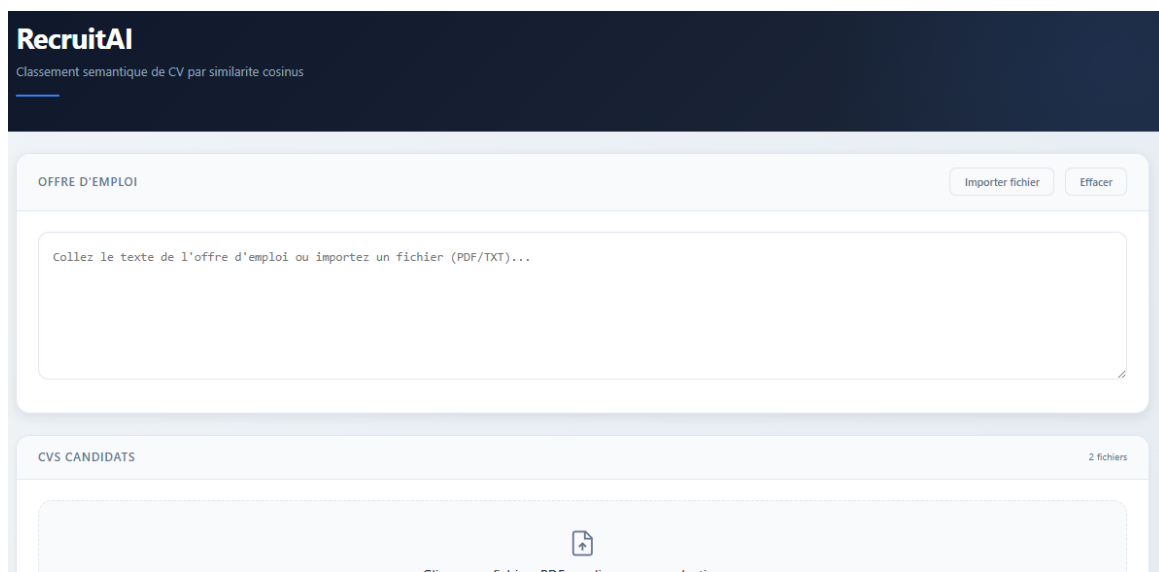
Le moteur IA est constitué des modules d'extraction de texte et de chunking. L'intégration dans le backend se fait par :

1. Importation : en tête du fichier principal, on importe les fonctions nécessaires.
2. Appel unique : dans la route `/upload`, on appelle ces fonctions avec l'offre et les fichiers.
3. Gestion des erreurs : si une exception est levée par le moteur, le serveur renvoie une réponse structurée indiquant la nature du problème.

4.4 Interface utilisateur

4.4.1 Structure de la page

La page web unique (`index.html`) se compose de trois zones principales, illustrées par les figures 19 et 20.



The screenshot displays the RecruitAI web application interface. At the top, a dark blue header contains the logo 'RecruitAI' and the tagline 'Classement sémantique de CV par similarité cosinus'. Below the header, the main content area is divided into two primary sections. The first section, titled 'OFFRE D'EMPLOI', features a large text input field with the placeholder text 'Collez le texte de l'offre d'emploi ou importez un fichier (PDF/TXT)...'. To the right of this field are two buttons: 'Importer fichier' and 'Effacer'. The second section, titled 'CVS CANDIDATS', shows a file upload area with a dashed border and a central icon of a document with an upward arrow. Below this icon is the text 'Cliquez sur un fichier PDF ou cliquez pour sélectionner'. In the top right corner of this section, it indicates '2 fichiers'.

FIGURE 19 – Formulaire d'analyse : saisie de l'offre d'emploi et upload des CVs.

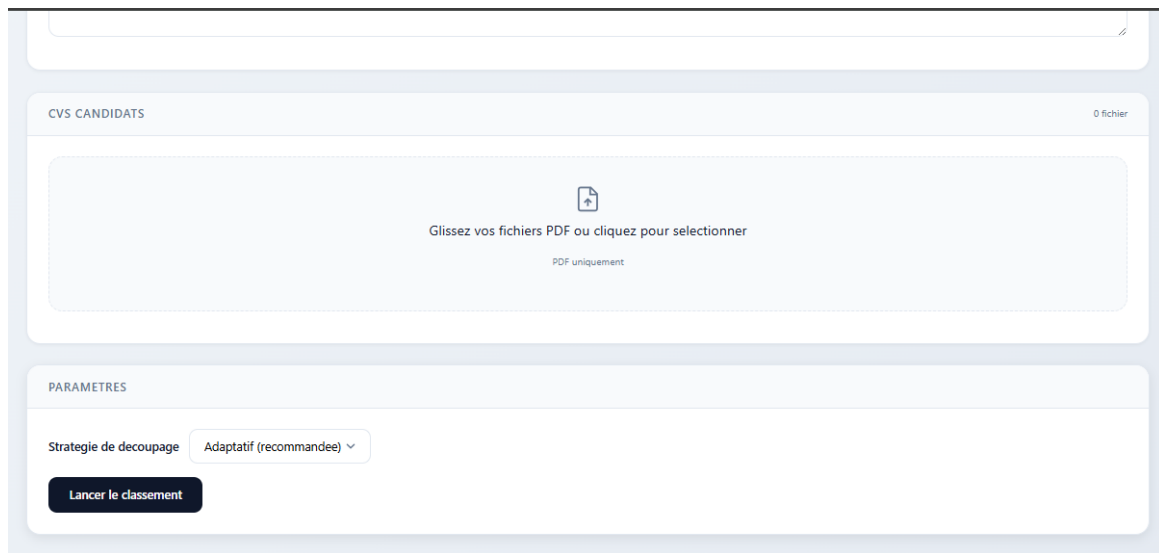


FIGURE 20 – Zone paramètres et résultats.

Les trois zones sont les suivantes :

1. **Zone offre d'emploi** : un champ texte permettant de coller l'offre ou d'importer un fichier (PDF ou TXT).
2. **Zone CVs candidats** : une zone de drag & drop pour importer des fichiers PDF, avec affichage des fichiers sélectionnés.
3. **Zone paramètres et résultats** : un sélecteur de stratégie de chunking, un bouton de lancement, et une zone d'affichage des résultats.

4.4.2 Comportement dynamique – JavaScript

Lorsque l'utilisateur clique sur Lancer le classement, le script JavaScript effectue les opérations suivantes :

1. Récupère le contenu de l'offre et la liste des fichiers.
2. Construit un objet FormData et envoie une requête fetch vers /upload.
3. Pendant l'attente, le bouton est désactivé et un indicateur de chargement apparaît.
4. À réception de la réponse JSON, il parcourt le tableau results et génère pour chaque candidat une carte HTML présentant :
 - le nom du candidat,
 - le score global (pourcentage) avec une barre de progression,
 - le nombre de blocs analysés et la position du bloc décisif.
5. Les cartes sont insérées dans le conteneur prévu, et le bouton est réactivé.

4.4.3 Fiche détaillée du candidat

En cliquant sur une carte, une fenêtre modale s'ouvre, présentant la fiche détaillée du candidat (Figure 22).

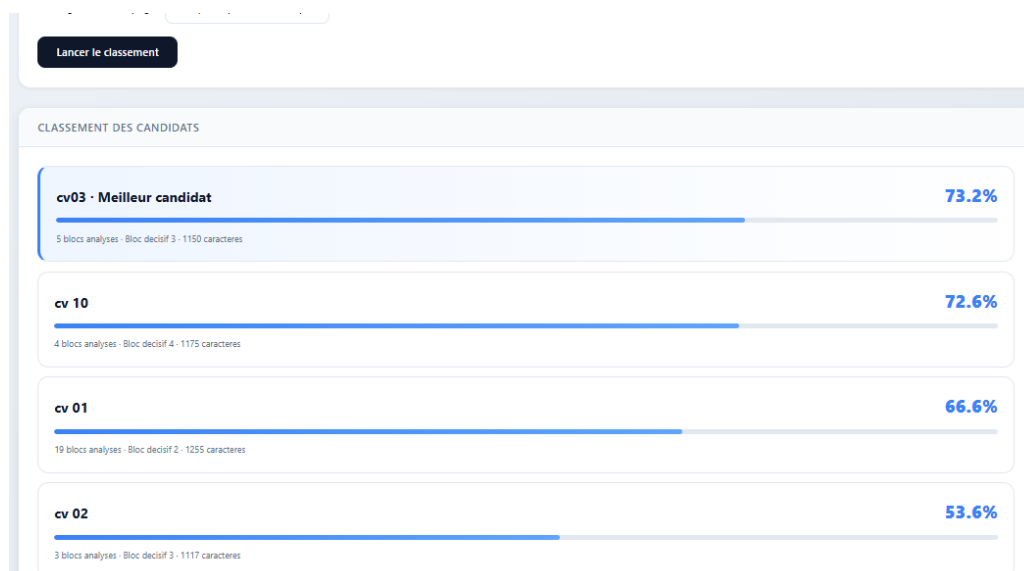


FIGURE 21 – Visualisation du résultat de classement

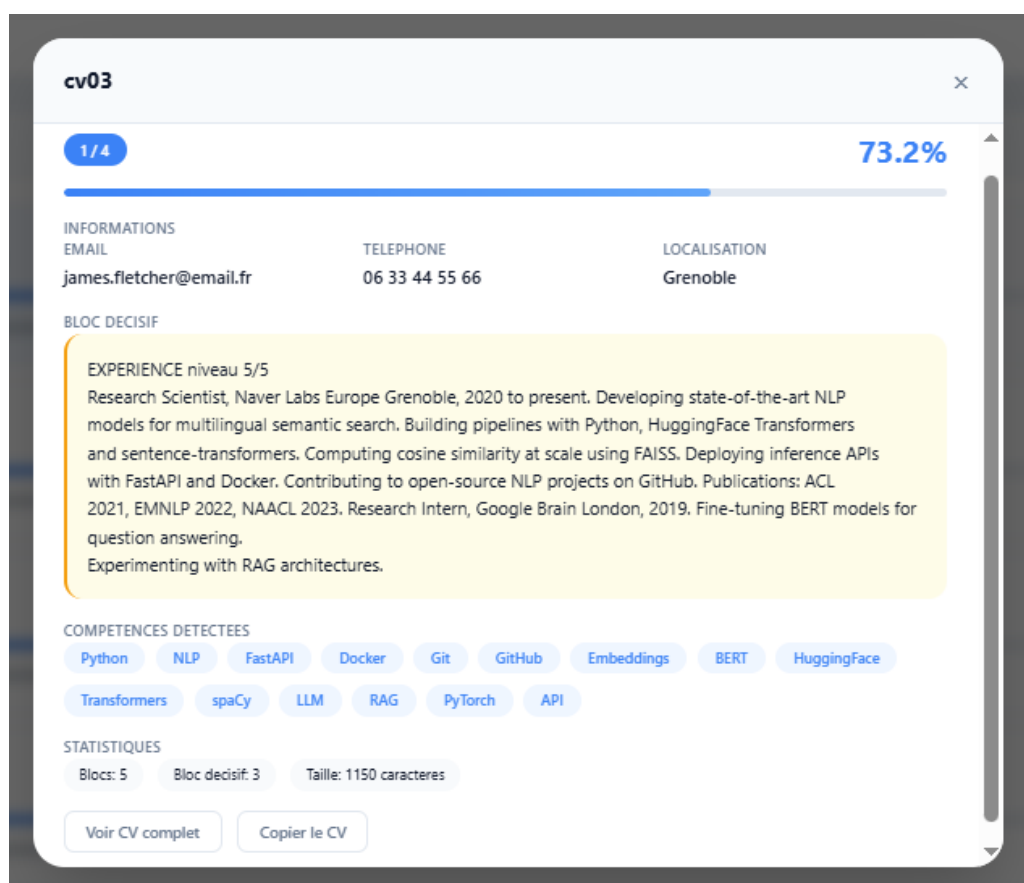


FIGURE 22 – Fiche détaillée d'un candidat.

La fiche contient les informations suivantes :

- **Score et classement** : score global du candidat et sa position dans le classement.
- **Informations personnelles** : email, téléphone, localisation, profil LinkedIn (extraits automatiquement par expressions régulières).
- **Bloc décisif** : extrait du CV qui a obtenu le meilleur score de similarité avec l'offre.
- **Compétences détectées** : liste des mots-clés techniques identifiés dans le CV.

- **Statistiques** : nombre de blocs analysés, position du bloc décisif, taille du CV en caractères.
- **Actions** : boutons pour visualiser le CV complet et copier le texte.

4.4.4 Visualisation du CV complet

Le bouton Voir CV complet ouvre une nouvelle fenêtre modale (Figure 23) affichant le texte intégral du CV extrait. Cette fonctionnalité permet au recruteur de vérifier manuellement les informations du candidat si nécessaire.

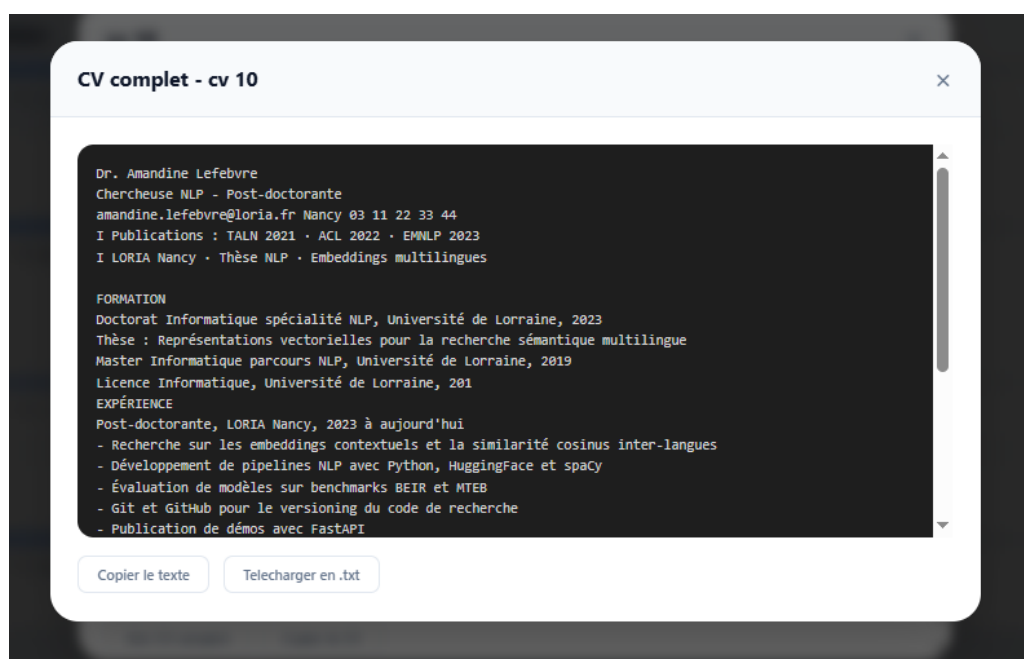


FIGURE 23 – Modale de visualisation du CV complet.

Dans cette modale, le recruteur peut :

- Copier l'intégralité du texte dans le presse-papier,
- Télécharger le texte au format TXT pour un archivage local.

4.5 Extraction automatique d'informations

Afin d'enrichir la fiche candidat, le système extrait automatiquement certaines informations à partir du texte du CV, à l'aide d'expressions régulières (regex) :

- **Email** : détection d'une chaîne au format `nom@domaine.com`.
- **Téléphone** : détection de numéros français (10 chiffres, espaces éventuels).
- **LinkedIn** : détection d'URL `linkedin.com/in/....`
- **Localisation** : détection des grandes villes françaises (Paris, Lyon, Marseille, etc.).

4.6 Tests et validation

4.6.1 Environnement de test

Les tests ont été réalisés sur une machine locale (Windows 11, Python 3.11) avec un jeu de 20 CVs factices, couvrant des profils variés (NLP, DevOps, Data Science) et différents formats de PDF.

4.6.2 Scénarios fonctionnels

Le Tableau 9 résume les principaux tests exécutés et leurs résultats.

Scénario	Résultat
Envoi d'un seul CV (PDF textuel)	Score calculé, affichage correct
Envoi de 10 CV simultanés	Classement complet en moins de 15 secondes
Offre vide	Message d'erreur Veillez renseigner une offre
Fichier non PDF	Fichier ignoré, analyse continue
Changement de stratégie	Scores recalculés avec la nouvelle stratégie
Clic sur un candidat	Fiche détaillée affichée
Bouton Voir CV complet	Modale avec texte intégral
Fermeture modale	Clic en dehors ou touche Échap

TABLE 9 – Résultats des tests fonctionnels.

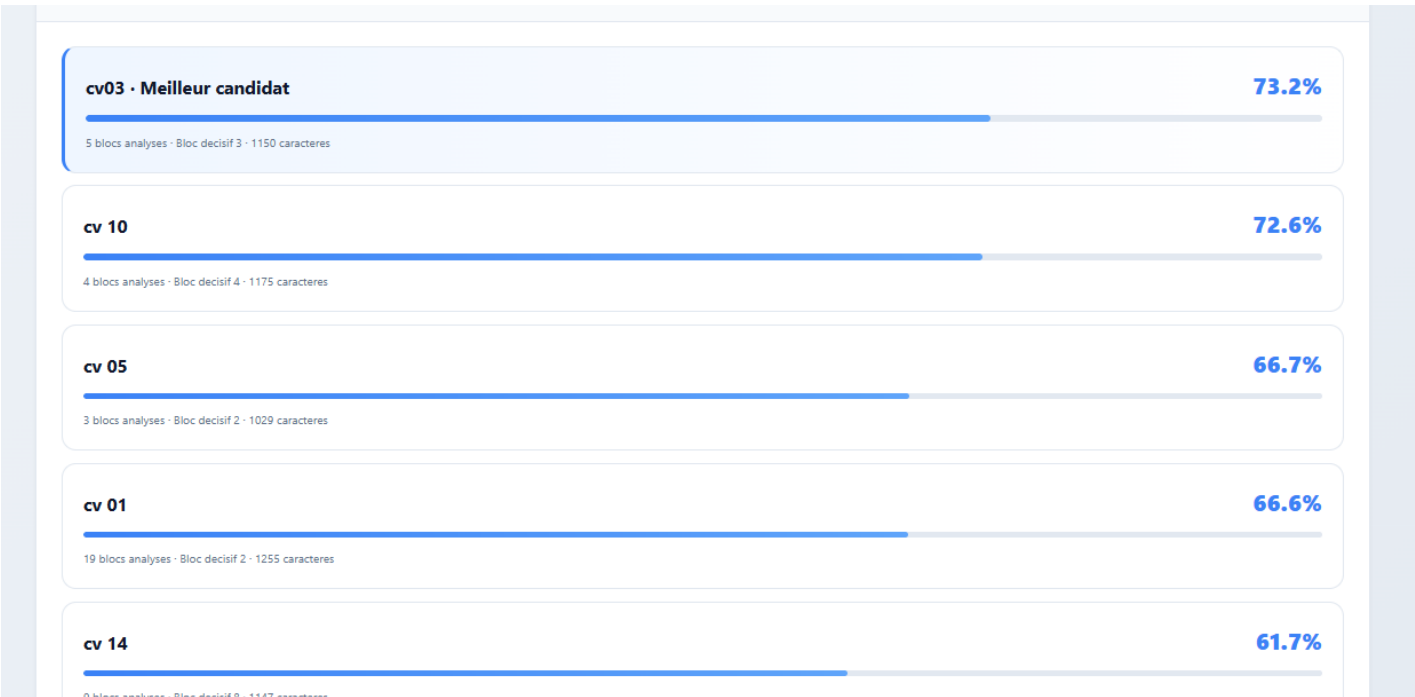


FIGURE 24 – Exemple de classement obtenu après analyse des CVs.

Comme l'illustre la Figure 24, le classement s'affiche correctement avec les scores et l'iden-

tification du meilleur candidat. L'ensemble des tests a validé le bon fonctionnement de l'application.

4.7 Conclusion

Ce chapitre a présenté l'interface web développée pour le système de matching CV/offre. L'architecture client-serveur choisie permet de séparer clairement les préoccupations : le frontend gère l'interaction utilisateur et le rendu, tandis que le backend orchestre l'exécution du moteur IA.

Les tests réalisés sur un jeu de CVs factices montrent que l'application fonctionne correctement dans un environnement local. Les principales fonctionnalités (import d'offre, upload de CVs, classement par similarité, inspection détaillée des candidats) sont opérationnelles et répondent aux besoins identifiés dans le diagramme de cas d'utilisation.

Conclusion générale

Bilan du travail réalisé

Ce travail de TER avait pour objectif de concevoir et développer un système automatique de matching entre des CVs et des offres d'emploi, basé sur des méthodes de traitement automatique du langage naturel et d'apprentissage profond.

Nous avons commencé par étudier les notions fondamentales nécessaires à la compréhension du projet : les embeddings de texte, la similarité cosinus, les Transformers et les sentence embeddings. Cette étude théorique nous a permis de justifier nos choix techniques.

Dans un second temps, nous avons abordé la problématique de l'extraction de texte à partir de CVs au format PDF. Nous avons implémenté une solution basée sur PyMuPDF, capable de gérer des mises en page complexes (colonnes, puces, barres de progression) et d'appliquer un nettoyage approfondi du texte extrait.

Le cœur du système repose sur l'évaluation et la comparaison de six stratégies de chunking, de la simple segmentation par sauts de ligne à une approche hiérarchique adaptative avec fusion sémantique. Les résultats expérimentaux montrent que la stratégie adaptative est la plus robuste, obtenant les meilleures performances sur quatre des cinq métriques évaluées (NDCG, Spearman, Precision@3, MRR).

Enfin, nous avons développé une interface web complète avec Flask, offrant une expérience utilisateur intuitive : import de l'offre, upload des CVs par drag & drop, classement dynamique, fiches détaillées et visualisation du texte extrait.

Limites et perspectives

Malgré les résultats satisfaisants, certaines limites subsistent. L'extraction de texte reste perfectible pour les PDFs très complexes ou scannés, et le chunking adaptatif, bien que performant, a un coût de calcul plus élevé que les stratégies simples.

Plusieurs perspectives d'amélioration sont envisageables :

- **Export des résultats** : ajouter la possibilité d'exporter le classement aux formats CSV ou PDF.
- **Filtrage** : permettre au recruteur de fixer un seuil de score minimum pour préqualifier les candidats.
- **Base de données** : sauvegarder les analyses précédentes pour un historique.
- **Traitement batch** : permettre l'upload d'un dossier entier de CVs.
- **Amélioration du chunking** : entraîner un modèle spécifique pour la détection des frontières sémantiques.

Retour d'expérience

Ce projet nous a permis de mettre en pratique des concepts théoriques du NLP et du deep learning, tout en développant une application complète du backend jusqu'à l'interface utilisateur. La difficulté principale a été de trouver le bon équilibre entre la qualité du matching et les performances de calcul, notamment lors du choix des stratégies de chunking.

L'utilisation de Flask et de sentence-transformers s'est révélée particulièrement adaptée pour ce type de projet, offrant un bon compromis entre simplicité de développement et efficacité.

Références

- [1] Wikipédia, *Apprentissage automatique*.
https://fr.wikipedia.org/wiki/Apprentissage_automatique
- [2] Aspexit, *Réseau de neurones : on va essayer de démystifier un peu tout ça*.
<https://aspexit.com/fr/blog/reseau-de-neurones-on-va-essayer-de-demystifier-un-peu-tout-ca-1>
- [3] Suresh Beekhani, *Understanding Forward and Backward Propagation in Neural Networks*, LinkedIn Pulse.
<https://www.linkedin.com/pulse/understanding-forward-backward-propagation-neural-suresh-beekhani-e0rkf>
- [4] Anthropic, *Claude* (version Sonnet 4.5), outil d'intelligence artificielle générative, 2026.
- [5] Wikimedia Commons, *Self-Attention (Scaled dot-product Attention)*.
[https://commons.wikimedia.org/wiki/File:Self-Attention_\(Scaled_dot-product_Attention\).png](https://commons.wikimedia.org/wiki/File:Self-Attention_(Scaled_dot-product_Attention).png)
- [6] Kenneth W. Church, *Word2Vec*, Natural Language Engineering, vol. 23, no. 1, pp. 155–162, 2017.
<https://doi.org/10.1017/S1351324916000334>
- [7] Jeffrey Pennington, Richard Socher et Christopher D. Manning, *GloVe : Global Vectors for Word Representation*, Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2014.
<https://nlp.stanford.edu/projects/glove/>
- [8] Ashish Vaswani et al., *Attention Is All You Need*, arXiv, 2017.
<https://arxiv.org/pdf/1706.03762>
- [9] Aspose Pty. Ltd., *pypdf | Bibliothèque Python pour la fusion et la division de PDF*, Aspose Pty. Ltd., 25 juin 2025.
<https://products.documentprocessing.com/fr/merger/python/pypdf/>
- [10] Bispo, N., *pdfplumber : A Guide to PDF Text and Table Extraction*, Unstract.com, 28 octobre 2025.
<https://unstract.com/blog/guide-to-pdfplumber-text-and-table-extraction-capabilities/>
- [11] PyMuPDF, *Introduction — PyMuPDF 1.21.0 documentation*.
<https://pymupdf-test.readthedocs.io/en/latest/index.html>
- [12] Ngongz. (n.d.). *PyPDF or PyMuPDF ?* Reddit. Disponible à : https://www.reddit.com/r/learnpython/comments/1796l3g/pypdf_or_pymupdf/
- [13] Sentence-Transformers, *paraphrase-multilingual-MiniLM-L12-v2*, <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>
- [14] Lucid.app, figures créées pour ce mémoire. https://lucid.app/lucidspark/3d787838-b448-4125-8f9f-043ffdc56b7/edit?viewport_loc=-422%2C1676%2C9406%2C4180%2CIIRQCcytBaDcA&invitationId=inv_00b8bd94-0181-4ef4-a86f-eb49becfec67